*Software: Practice and Experience*

WILEY

Allergy EUROPEAN JOURNAL OF ALLERGY AND CLINICAL IMMUNOLOGY EAACI

RESEARCH ARTICLE OPEN ACCESS

# SecChaos-RL: An Autonomous Threat-Aware Resilience Engineering Framework for Microservices Under Security Chaos

Thurbilli Karthik[1] | Michael Arock[1] | Vinay Raj[2]

[1]Department of Computer Applications, National Institute of Technology, Tiruchirappalli 620015, Tamil Nadu, India | [2]Department of Computational & Data Science, National Institute of Technology, Warangal 506004, Telangana, India |

**Correspondence:** Thurbilli Karthik (405124053@nitt.edu)

**Received:** 20 January 2026 | **Revised:** 21 January 2026 | **Accepted:** 22 January 2026

**Funding:** The authors received no external funding for this study.

**Keywords:** Security Chaos Engineering | Microservice Resilience | Reinforcement Learning | Self-Adaptive Systems | Fault Injection | DevSecOps

## ABSTRACT

Distributed cloud-native microservices increase operational attack surfaces and create complex dependency failure cascades across services. Although Security Chaos Engineering (SCE) uncovers vulnerabilities through proactive fault injection, current practices remain manual, declarative, and open-loop. Automating runtime remediation introduces an operational dilemma: aggressive mitigations, such as container restarts and network isolation, often cause cascading timeouts and service outages. This paper presents SecChaos-RL, an autonomous closed-loop resilience framework combining SCE with deep reinforcement learning. Formulated as a multi-objective Markov Decision Process, the framework balances security risk reduction, service availability, Mean Time to Recovery (MTTR), and mitigation cost. The neural policy uses Multi-Head Self-Attention over dependency telemetry, Threat-Aware Feature Gating, an Auxiliary Risk Prediction Head, and Threat-Adaptive Temperature Exploration. Across 30 independent random seeds under multi-stage security chaos, empirical results demonstrate critical operational trade-offs: although static alert rules achieve higher local risk reduction (55.5%), they cause severe action churn (30.8% false mitigation rate). SecChaos-RL achieves balanced multi-objective stability, recovering significantly faster under cascading failure than conservative heuristics (MTTR = 1.79 steps versus 2.29 for Threshold-Based, $p = 0.0013$), halving unnecessary dispatches (20.9% false mitigation rate), and deciding actions within 0.44 ms. The framework establishes practical deployment boundaries between deterministic heuristics and reinforcement learning control.

## 1 | Introduction

Modern enterprise applications in banking, healthcare, retail, and telecommunications rely heavily on distributed microservice architectures deployed on cloud platforms [5, 1, 33]. Unlike monolithic architectures, microservices split business functionality into small, loosely coupled services that interact through lightweight protocols such as REST or gRPC and message brokers [36, 8]. While this decomposition helps software teams deploy features independently, it increases runtime complexity. Real-world applications often involve deep dependency graphs where security breaches and operational faults can cascade rapidly across services [12, 22].

Maintaining reliability and security in large distributed environments is an ongoing challenge in software engineering [7]. Standard pre-production practices, including static analysis (SAST), dynamic scanning (DAST), automated test plan generation [15], buffer overflow vulnerability detection [17], bug report classification [11], and opcode feature learning [37], help identify defects during build time. However, pre-deployment checks inspect components in isolation and cannot capture runtime dynamics, such as active cyber attacks, configuration drift, and cascading performance slowdowns under production load [31, 25]. Studies on

adversarial attacks against software analytics models [3] further emphasize the need to evaluate defensive controls under live adversarial conditions.

To build confidence in distributed systems, practitioners use *Chaos Engineering*, which intentionally introduces faults to test system resilience [5, 27]. Early tools such as Netflix's Chaos Monkey and newer container frameworks, including ChaosBlade [10], LitmusChaos [18], Gremlin [14], Chaos Mesh [9], and AWS Fault Injection Simulator [2], inject infrastructure faults like container terminations, network delays, and resource spikes [8, 12, 4]. *Security Chaos Engineering* (SCE) applies this concept to security validation [31, 23]. Instead of physical crashes alone, SCE introduces security-related faults, such as expired TLS certificates, vulnerable libraries, leaked tokens, privilege misconfigurations, and exposed internal endpoints, to assess whether defensive safeguards function under attack [23, 6].

A key limitation of current SCE practices is that they remain mostly manual and open-loop [25, 31]. Engineers execute preplanned fault scripts, review monitoring dashboards afterwards, and manually apply fixes. In continuous-deployment DevSecOps pipelines, manual remediation is often too slow to block active attacks that exploit short vulnerability windows [31, 19].

Automating security remediations in live microservices introduces two primary operational challenges:

- **Trade-off Between Security and Availability:** Applying prompt security remediations, such as killing suspicious containers, blocking network traffic, revoking credentials, or rolling back releases, can cause severe service disruptions. In interconnected microservices, shutting down an authentication or catalog service causes timeouts in calling services, trips circuit breakers, and drops active customer requests, turning a local security problem into a widespread outage [1, 29].
- **Deficiencies of Static Rules:** Operations teams commonly rely on simple alert-matching rules (for example, restarting a service whenever an alert fires). Under concurrent faults and heavy user traffic, such rules trigger alert flapping and mitigation storms. Multiple services run uncoordinated fixes simultaneously, consuming CPU and memory, increasing latency, and lowering overall throughput [29].

Self-adaptive software systems (SASS) offer a structured approach for closed-loop recovery using the MAPE-K (Monitor, Analyze, Plan, Execute, Knowledge) control model [28, 13, 29]. Reinforcement learning (RL) is well suited for the Planning step, allowing an agent to learn mitigation policies by interacting with the target environment [35, 20]. While deep reinforcement learning (DRL) has shown strong performance across complex decision benchmarks [32, 21, 38, 26], standard RL algorithms (such as DQN [20], A2C [21], and baseline PPO [30]) perform poorly when applied directly to microservice security. Standard policies treat telemetry dimensions uniformly, ignore dependency graph structures, struggle with delayed remediation execution, and risk triggering erratic exploratory actions during security spikes [41, 34, 24, 40]. In addition, recent reviews of Large Language Models (LLMs) in software engineering [16] note that static design recommendations cannot guarantee runtime service safety without closed-loop verification.

## 1.1 | Research Questions

To address these technical challenges and deliver a practically useful software contribution, this paper formulates and investigates four core Research Questions (RQs):

- **RQ1 (Resilience and Recovery Efficacy):** How effectively does SecChaos-RL reduce systemic security risk and minimize mean time to recovery (MTTR) compared to deterministic software engineering baselines (No-Mitigation, Static Rule-Based, Threshold-Based) and standard RL algorithms (DQN, A2C, PPO)?
- **RQ2 (Operational Availability and Trade-Off Balance):** How well does SecChaos-RL preserve live microservice availability and prevent false mitigation storms when subjected to concurrent security faults and cascading dependency failures?
- **RQ3 (Architectural Ablation and Component Contributions):** What are the isolated contributions of the individual framework components—multi-objective reward design, multi-head self-attention, auxiliary risk prediction, threat-aware gating, and adaptive policy temperature—to overall system survivability?
- **RQ4 (Computational Overhead and Practical Feasibility):** What is the runtime decision latency and computational overhead of SecChaos-RL, and does it satisfy the real-time execution constraints required for cloud-native microservice orchestration?

## 1.2 | Main Contributions

This paper makes the following concrete contributions to the software engineering and microservice resilience literature:

1. **Software Engineering Framework Contribution:** We design and implement SecChaos-RL, an open, modular Python framework that operationalizes closed-loop Security Chaos Engineering. The architecture decouples continuous Poisson fault injection (`ChaosEngine`), service dependency DAG propagation (`DependencyTopology`), threat-aware decision orchestration (`ThreatAwareOrchestrator`), asynchronous mitigation action execution with queuing and transient delays (`MitigationExecutor`), and reproducible statistical benchmarking (`BenchmarkHarness`).
2. **Algorithmic Reinforcement Learning Contribution:** We formalize the microservice mitigation problem as a multi-objective Markov Decision Process (MDP). To address the challenges of correlated telemetry and delayed execution feedback, we introduce a domain-tailored neural policy architecture incorporating Multi-Head Self-Attention over dependency features, Threat-Aware Feature Gating (amplifying vulnerability signals during active compromise), an Auxiliary Risk Prediction Head (regularizing latent representations against future state transitions), and Threat-Adaptive Policy Temperature Exploration.
3. **Formal Algorithmic Workflows:** We provide complete, mathematically rigorous algorithmic workflows for both closed-loop runtime mitigation execution (Algorithm 1) and threat-aware policy optimization with auxiliary risk regularization (Algorithm 2).

4. **Rigorous Comparative Evaluation Across 30 Seeds:** We conduct an extensive empirical evaluation across 30 independent random seeds, benchmarking SecChaos-RL against deterministic software heuristics (No-Mitigation, Static Rule-Based, Threshold-Based, and Queue-Aware Cooldown heuristics) and standard RL algorithms (DQN, A2C, standard PPO). Using repeated-measures ANOVA and paired hypothesis testing, we rigorously characterize the operational trade-offs between heuristic rule triggering and reinforcement learning control.
5. **Six-Stage Ablation and Hardware Profiling:** We execute a six-stage ablation study analyzing the incremental operational behavior and trade-offs of each architectural component (reward shaping, self-attention, auxiliary risk prediction, threat-aware gating, and adaptive temperature exploration), and profile runtime decision latency ($0.44 \pm 0.15$ ms per-step benchmark latency; $0.46 \pm 0.52$ ms mean, 0.36 ms median, 1.04 ms $p_{99}$ micro-benchmark on CPU) to assess practical deployability in cloud-native container environments.
6. **Complete, Verifiable Replication Package:** We provide a self-contained, open-source replication package containing the entire simulator environment, agent implementations, evaluation scripts, raw CSV telemetry across all 30 seeds, and automated analysis code to ensure full reproducibility. All artifacts are provided with this submission as Supplementary Material and staged for open archiving upon publication.

The remainder of this paper is structured as follows. Section 2 reviews related literature in chaos engineering, microservice resiliency benchmarking, and self-adaptive security. Section 3 presents the SecChaos-RL software framework architecture. Section 4 formalizes the threat model, MDP formulation, algorithms, and deep policy design. Section 5 details the experimental setup and evaluation protocol. Section 6 presents the empirical results and answers RQ1–RQ4. Section 7 discusses practical implications for software engineers. Section 8 addresses threats to validity, and Section 9 concludes the paper.

## 2 | Background and Related Work

This section reviews related research across four key areas: Chaos Engineering, Microservice Resiliency Benchmarking, Security Chaos Engineering, and Autonomous Reinforcement Learning for Cyber Defense.

### 2.1 | Chaos Engineering and Distributed Fault Injection

Chaos Engineering was established as a disciplined practice by Netflix engineers to verify the reliability of large-scale distributed cloud applications [5]. Rosenthal and Jones synthesized core industry practices, emphasizing the necessity of automating experiments, minimizing blast radiuses, and running continuous turbulence campaigns in production [27]. A comprehensive multi-vocal literature review by Owotogbe et al. [25] analyzed 96 academic and industrial sources, formalizing a taxonomy of chaos engineering platforms across execution environments, automation modes, and target architectures. Esen et al. [12] systematically reviewed chaos experiments in microservices across 31 papers and 38 tools, noting that mainstream tooling (e.g., Chaos Mesh [9], Gremlin [14], ChaosBlade [10], LitmusChaos [18], and AWS FIS [2]) focuses overwhelmingly on physical and network faults—such as packet latency, CPU stress, and container restarts—while neglecting security-specific compromise states. Bagehorn et al. [4] demonstrated fault injection platforms for training AIOps models, yet focused on operational anomalies rather than adversarial attack mitigations.

Addressing the need for structured, configurable fault injection platforms, Camacho et al. [8] introduced *Chaos as a Software Product Line* (CoSPL) in *Software: Practice and Experience*. Their platform, *Pystol*, models fault injection capabilities as software product line features across open hybrid-cloud environments (e.g., OpenShift and Kubernetes). While CoSPL demonstrates the software engineering value of configurable chaos architectures, its execution remains declarative and open-loop: it verifies system tolerance against pre-scripted variations but lacks an autonomous, learning-based runtime mitigation engine.

### 2.2 | Microservice Resiliency Benchmarking and Self-Healing

In distributed microservice systems, resilience patterns—such as timeouts, retries, fallbacks, and circuit breakers—are standard software engineering mechanisms to mitigate transient degradations [1, 29, 33]. Taibi et al. [36] analyzed microservice API evolution patterns in *Software: Practice and Experience*, emphasizing that service interface drift and asynchronous call coupling create fragile dependency chains.

In *Software: Practice and Experience*, Aderaldo et al. [1] presented *ResilienceBench*, a declarative benchmarking approach and Kubernetes operator tool designed for the controlled evaluation of microservice resiliency patterns under simulated failures (tested on Google Online Boutique). Their work demonstrates that naive retry configurations amplify cascading service failures and increase tail latency during downstream disruptions.

Self-healing systems literature provides the broader control-theoretic foundation for autonomous software resilience. In *Software: Practice and Experience*, Schneider, Barker, and Dobson [29] categorized multi-tier self-healing frameworks using the autonomic MAPE-K control loop [13, 28]. They highlighted that while rule-based self-healing loops succeed in isolated components, they exhibit severe stability issues when managing multi-tier dependencies subject to concurrent, overlapping disturbances. SecChaos-RL directly builds upon this insight: rather than using static rule heuristics that fight one another, it employs a centralized, threat-aware RL policy to coordinate multi-service mitigations.

### 2.3 | Security Chaos Engineering (SCE) and DevSecOps

Security Chaos Engineering applies the empirical philosophy of chaos experimentation directly to cyber resilience [31]. Myrbakken and Colomo-Palacios [22] reviewed security bottlenecks in DevSecOps pipelines, concluding that traditional pre-production testing fails to keep pace with continuous deployment agility. In subsequent work, Myrbakken et al. [23] formalized SCE

as a continuous assessment paradigm, demonstrating how automated security fault injection exposes configuration drifts and access control weaknesses.

Shortridge and Rinehart [31] detailed the principles of SCE, arguing that complex software systems are fundamentally nondeterministic and that resilience depends on validating system behavior under active attack conditions. Bedoya et al. [6] demonstrated the integration of SCE with Large Language Models (LLMs) to discover architectural threat vectors during the design phase of a commercial retail application. However, existing SCE tools remain limited to passive vulnerability discovery: they detect when a system fails a security property, but rely entirely on human engineers to craft and execute mitigations [31, 25].

### 2.4 | Reinforcement Learning for Cyber Defense and Mitigation

Reinforcement learning has demonstrated significant capability in complex decision-making domains [32, 35]. Value-based methods (e.g., Deep Q-Networks [20]) and policy gradient approaches (e.g., Advantage Actor-Critic [21] and Proximal Policy Optimization [30]) have enabled autonomous agents to learn sophisticated control strategies in high-dimensional state spaces.

In cybersecurity, deep reinforcement learning (DRL) has been applied to intrusion detection, cyber-physical defense, IoT protection [40], and autonomous penetration testing [24, 19]. Standen et al. [34] introduced *CybORG*, a Gym-based simulation environment for training autonomous cyber defense agents in Security Operations Centers (SOCs). However, CybORG assumes static enterprise network topologies and fixed IP routing graphs, omitting microservice dependency cascades, runtime HTTP/gRPC request availability, and chaos-driven operational trade-offs. Algorithmic enhancements, such as Reward-based Prioritization PPO (RP-PPO) by Zheng et al. [41], have improved sample efficiency, but remain unspecialized for the dual constraints of security isolation and service continuity. Standard frameworks such as Gymnasium [38] and Stable-Baselines3 [26] provide standardized algorithmic baselines, which we leverage and extend for microservice security control.

### 2.5 | Critical Synthesis and Identified Gap

As summarized in Table 1, existing literature is divided into two disconnected silos: (1) chaos engineering frameworks that inject turbulence but rely on manual remediation, and (2) RL cyber defense agents that operate in static, abstract network simulations without considering microservice availability SLAs. **SecChaos-RL bridges Security Chaos Engineering with deep reinforcement learning to provide closed-loop, threat-aware microservice resilience.**

## 3 | SecChaos-RL Framework Architecture

SecChaos-RL is designed as an extensible, closed-loop software architecture implementing the classic MAPE-K self-adaptation paradigm [28, 13]. The framework operates in discrete operational time steps, continuously observing telemetry from monitored microservices, injecting controlled security chaos, inferring optimal mitigation decisions, and asynchronously executing remediations.

As illustrated in Figure 1, SecChaos-RL is composed of five decoupled, modular software components:

1. **Monitored Microservice Topology (`DependencyTopology`):** Represents the operational target system structured as a directed acyclic graph (DAG) $\mathcal{G} = (\mathcal{V}, \mathcal{E})$. Each node $v \in \mathcal{V}$ models an individual microservice maintaining local security risk $r_t(v) \in [0, 1]$ and service availability $a_t(v) \in [0, 1]$. Directed edges $(u, v) \in \mathcal{E}$ represent synchronous downstream API call dependencies (e.g., the Gateway depends on Authentication, which depends on the User service).
2. **Security Chaos Engine (`ChaosEngine`):** Implements dynamic, stochastic security fault injection across eight standardized vulnerability categories. The engine generates faults according to configurable Poisson arrival processes and assigns random severities and operational durations. It models active attacker exploitation by tracking concurrent attack sessions that target vulnerable services.
3. **Telemetry and State Monitor (`StateMonitor`):** Continuously inspects the microservice cluster, aggregating individual service risks, resource metrics (CPU utilization, memory consumption, network overhead), transaction success ratios, and MTTR tracking into a standardized 16-dimensional observation vector $o_t \in \mathbb{R}^{16}$.
4. **Threat-Aware Decision Orchestrator (`ThreatAwareOrchestrator`):** The intelligence engine of the framework, implementing our specialized deep actor-critic policy. It ingests the observation vector, computes multi-head self-attention across features, gates security dimensions based on active threat severity, and selects a mitigation action $a_t \in \mathcal{A}$.
5. **Asynchronous Mitigation Executor (`MitigationExecutor`):** Translates abstract policy actions into operational operations (e.g., certificate rotation, dependency patching, RBAC policy restriction). The executor explicitly models operational delays: actions are placed in an asynchronous execution queue, requiring $\tau \in \{0, 1, 2\}$ operational steps to complete, while consuming transient CPU and memory resources during deployment. Upon completion, the executor actively invokes service healing in the dependency graph, reducing service risk and restoring availability.

## 4 | Threat Model and Decision Methodology

### 4.1 | Threat Model and Fault Taxonomy

We assume an adversarial environment where an external attacker continuously probes public-facing entry points (e.g., API Gateway) and attempts lateral movement across internal microservices. The attacker seeks to exploit unpatched dependencies, leaked secrets, or misconfigured access permissions to compromise data integrity, escalate privileges, and exhaust system resources.

Table 2 formalizes the eight security fault categories modeled in SecChaos-RL, mapped to their target services. When a fault $f_i$ with severity $\sigma_i \in [0.35, 0.65]$ is injected into service $u \in \mathcal{V}$, the

**TABLE 1** | Systematic Qualitative Comparison Between SecChaos-RL and Existing Approaches

| Approach | Target Domain | Control Loop | Fault Scenarios | Mitigation Strategy | Key Limitation / Difference |
|---|---|---|---|---|---|
| Chaos Monkey [5] | Cloud VMs | Open-loop testing | Random instance crash | Passive termination | No security faults; relies on human recovery. |
| CoSPL (Pystol) [8] | Hybrid Cloud / K8s | Declarative SPL | Network latency, pod kill | Configured scripts | Verifies resilience statically; lacks adaptive mitigation. |
| ResilienceBench [1] | Microservices / K8s | Declarative operator | Network drop, crash | Static retry/breaker | Evaluates performance patterns; omits security threats. |
| Self-Healing Survey [29] | Multi-tier Web | Rule-based MAPE-K | Resource exhaustion | Restart, replication | Predates cloud-native microservices; lacks DRL. |
| CybORG [34] | Enterprise SOC | Gym RL (PPO/DQN) | Lateral movement | Host isolation | Static networks; ignores microservice SLAs and chaos. |
| LLM-SCE [6] | DevSecOps CI/CD | Design-time advice | Missing auth, secrets | Offline Jira recommendations | Build-time analysis; lacks continuous runtime control. |
| **SecChaos-RL (Proposed)** | **Cloud Microservices** | **Closed-loop DRL** | **8 Security Chaos Faults** | **Adaptive Multi-Action** | **Autonomous, threat-aware Pareto balancing of security, availability, and MTTR.** |

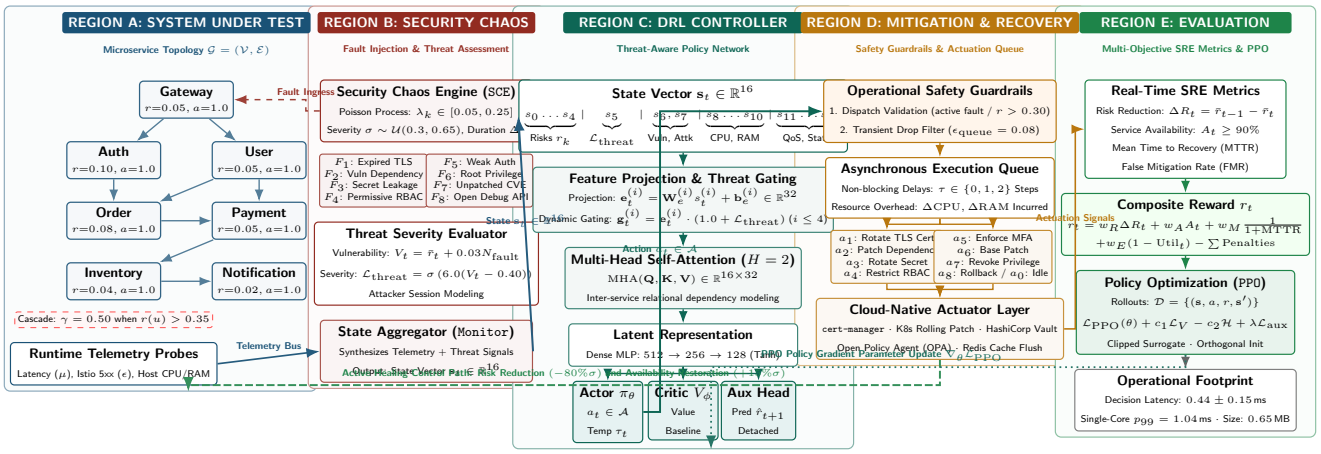



**FIGURE 1** | Overall SecChaos-RL closed-loop resilience engineering framework. The workflow proceeds through five coordinated functional regions: (Region A) Monitored microservice dependency graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with runtime telemetry monitoring; (Region B) Security Chaos Engine injecting standardized stochastic faults ($F_1$–$F_8$, Poisson arrival $\lambda_k \in [0.05, 0.25]$) alongside continuous logistic threat severity evaluation ($\mathcal{L}_{\text{threat}}$); (Region C) Threat-aware deep reinforcement learning controller featuring linear state feature projection, threat-aware feature gating, multi-head self-attention ($H = 2$), and three decoupled output heads (actor policy $\pi_\theta$ with adaptive temperature $\tau_t$, critic baseline $V_\phi$, and auxiliary risk prediction head $\hat{\mathbf{r}}_{t+1}$); (Region D) Operational safety guardrails (dispatch validation and transient queue drop filter $\epsilon_{\text{queue}} = 0.08$), asynchronous execution queue ($\tau \in \{0, 1, 2\}$), and cloud-native remediation actuators delivering active service healing (reducing targeted service risk by 80%$\sigma$ and restoring availability by +15%$\sigma$); and (Region E) Multi-objective performance evaluation tracking service availability, recovery time (MTTR), security risk reduction, and false mitigation rate (FMR), driving PPO policy gradient updates ($\nabla_\theta \mathcal{L}_{\text{PPO}}$) through the closed learning loop. Abbreviations: DAG: Directed Acyclic Graph; DRL: Deep Reinforcement Learning; FMR: False Mitigation Rate; MTTR: Mean Time to Recovery; PPO: Proximal Policy Optimization; SCE: Security Chaos Engineering.

**TABLE 2** | SecChaos-RL Security Fault Taxonomy

| ID | Fault Type | Target Service | Operational Impact |
|---|---|---|---|
| $F_1$ | Expired TLS Certificate | Gateway | Insecure channel; enables Man-in-the-Middle (MITM). |
| $F_2$ | Vulnerable Dependency | User | Known CVE in library; remote code execution risk. |
| $F_3$ | Secret / Token Leakage | User | Exposed credentials in log telemetry or environment. |
| $F_4$ | RBAC Misconfiguration | Authentication | Overly permissive role mappings; privilege escalation. |
| $F_5$ | Weak Authentication / MFA | Authentication | Brute-force susceptibility; bypassed credential checks. |
| $F_6$ | Excessive Privileges | Order | Service account running as root with unnecessary capabilities. |
| $F_7$ | Missing Security Patch | Inventory | Unpatched operating system or runtime vulnerability. |
| $F_8$ | Exposed Internal API | Gateway | Unauthenticated internal endpoint accessible externally. |

service's risk is immediately incremented:

$$r_{t+1}(u) \leftarrow \min(1.0, r_t(u) + \sigma_i), \quad a_{t+1}(u) \leftarrow \max(0.0, a_t(u) - 0.12 \cdot \sigma_i) \tag{1}$$

If the service's risk crosses the critical degradation threshold ($r_t(u) > 0.35$), cascading failure propagates along all outgoing

edges $(u, v) \in \mathcal{E}$:

$$r_{t+1}(v) \leftarrow \min\left(1.0, r_t(v) + \alpha_{casc} \cdot r_t(u) \cdot \gamma_{coupl}\right) \quad (2)$$

$$a_{t+1}(v) \leftarrow \max\left(0.0, a_t(v) - 0.12 \cdot \left[\alpha_{casc} \cdot r_t(u) \cdot \gamma_{coupl}\right]\right) \quad (3)$$

where $\alpha_{casc} = 0.22$ and $\gamma_{coupl} = 0.5$ represent the cascading propagation coefficient and inter-service coupling factor, respectively.

## 4.2 | Markov Decision Process (MDP) Formulation

The mitigation decision space is formalized as an infinite-horizon discounted Markov Decision Process $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$:

### 4.2.1 | Observation Space and Feature Specification

The agent receives a standardized 16-dimensional continuous observation vector $\mathbf{s}_t \in \mathbb{R}^{16}$:

$$\mathbf{s}_t = \left[r_t^{cert}, r_t^{dep}, r_t^{sec}, r_t^{rbac}, r_t^{auth}, \mathcal{L}_{threat}, \mathcal{V}_{score}, A_{act}^{norm}, u_{cpu}, u_{mem}, u_{net}, a_t, t_{rec}^{norm}, \bar{h}_{atk}, \eta_{prev}, |\mathcal{F}_{act}|^{norm}\right]^T \in [0,1]^{16} \quad (4)$$

Table 3 formalizes the complete 16-row specification of $\mathbf{s}_t$, detailing feature semantics, mathematical bounds, normalization functions, and telemetry sources.

The active fault count enters the observation vector in two distinct forms: in Index 15 as a normalized raw count ($|\mathcal{F}_{act}|/8.0 \in [0,1]$), and inside Index 6 ($\mathcal{V}_{score}$) as an additive compounding factor ($0.03|\mathcal{F}_{act}|$) added to mean risk $\bar{r}_t$. Both expressions are strictly bounded to $[0.0, 1.0]$. The continuous threat severity $\mathcal{L}_{threat} \in [0.0, 1.0]$ is computed via the logistic sigmoid gate:

$$\mathcal{L}_{threat} = \frac{1}{1 + \exp\left[-6.0\left(\mathcal{V}_{score} - 0.40\right)\right]} \quad (5)$$

### 4.2.2 | Action Space, Execution Queues, and Transition Semantics

The action space $\mathcal{A}$ comprises nine discrete operational mitigations:

$$\mathcal{A} = \{a_0 : \text{ignore/idle}, a_1 : \text{rotate_cert}, a_2 : \text{patch_dep}, a_3 : \text{rotate_secret}, a_4 : \text{restrict_rbac}, a_5 : \text{enable_mfa}, a_6 : \text{security_patch}, a_7 : \text{revoke_priv}, a_8 : \text{rollback}\} \quad (6)$$

Each mitigation action $a_k \in \mathcal{A}$ is parameterized by target fault codes, an asynchronous execution delay $d_k \in \{0, 1, 2\}$ steps, invocation cost $C(a_k)$, and transient resource overheads $(\Delta u_{cpu}, \Delta u_{mem}, \Delta u_{net})$:

- $a_0$ (ignore): Cost 0.0, delay $d_0 = 0$, overheads $(0.0, 0.0, 0.0)$, targets none.
- $a_1$ (rotate_cert): Cost 0.10, delay $d_1 = 1$, overheads $(0.02, 0.01, 0.01)$, targets $\{F1, F8\}$.
- $a_2$ (patch_dep): Cost 0.40, delay $d_2 = 2$, overheads $(0.15, 0.10, 0.05)$, targets $\{F2, F7\}$.
- $a_3$ (rotate_secret): Cost 0.20, delay $d_3 = 1$, overheads $(0.05, 0.03, 0.02)$, targets $\{F3\}$.
- $a_4$ (restrict_rbac): Cost 0.15, delay $d_4 = 1$, overheads $(0.03, 0.02, 0.01)$, targets $\{F4, F6\}$.
- $a_5$ (enable_mfa): Cost 0.10, delay $d_5 = 0$, overheads $(0.01, 0.01, 0.01)$, targets $\{F5\}$.
- $a_6$ (apply_patch): Cost 0.50, delay $d_6 = 2$, overheads $(0.20, 0.15, 0.05)$, targets $\{F7, F2\}$.
- $a_7$ (revoke_priv): Cost 0.20, delay $d_7 = 1$, overheads $(0.04, 0.03, 0.01)$, targets $\{F6, F4\}$.
- $a_8$ (rollback): Cost 0.35, delay $d_8 = 1$, overheads $(0.10, 0.08, 0.03)$, targets $\{F8, F1\}$.

**Asynchronous Queue and Operational Action Outcomes:** In live microservice operations, automated interventions interact with non-deterministic failure dynamics across three strictly **mutually exclusive** non-success operational states:

1. **Unnecessary Dispatch ($N_{wrong}$):** An action $a_t \neq 0$ is dispatched when neither an active fault matches its target codes nor the target service exhibits elevated vulnerability ($r_{target} \leq 0.30$). Such actions represent policy classification errors. They are immediately intercepted at the admission boundary, penalizing the agent ($\beta_4 \mathcal{P}_{wrong}$) and setting $\eta_{prev} \leftarrow 0.2$ without entering the queue $\mathcal{Q}$. FMR strictly measures this dispatch-level error: FMR $= N_{wrong} / \max(1, N_{actions})$.
2. **Transient Execution Failure ($N_{drop}$):** To model transient cloud-native RPC timeouts, pod scheduling delays, or admission control rejections, valid dispatches face an empirical 8% transient failure rate (92% successful queue admission). Dropped actions incur transient resource overheads $(\Delta u_{cpu}, \Delta u_{mem}, \Delta u_{net})$ and invocation cost without entering the queue $\mathcal{Q}$.
3. **Queue Redundancy ($N_{redundant}$):** An action was valid when dispatched and successfully admitted to $\mathcal{Q}$, but during its asynchronous queue delay $d_k$, an earlier maturing mitigation or natural fault expiration resolved the underlying fault. Upon maturity ($t_{due} = t + d_k$), the action executes but finds no matching active fault ($\mathcal{M}_a = \emptyset$). Because such actions were valid at dispatch, they represent unavoidable queue latency artifacts rather than controller misclassifications; they consume transient overheads without altering service health and are strictly distinct from $N_{wrong}$.

Transient resource overheads $(\Delta u_{cpu}, \Delta u_{mem}, \Delta u_{net})$ are applied immediately upon dispatch to cluster utilization (Section 3) regardless of admission success, accurately capturing the computational and network cost of command serialization, TLS handshakes, and container scheduler API requests.

Within each discrete simulation step $t$, operations proceed in the following strict order:

1. *Action Queue Processing & Dispatch:* Actions whose delay has expired ($t_{due} \leq t$) are dequeued and executed, invoking active service healing ($\Delta r = -0.80 \times severity$, restoring availability by $+0.15 \times severity$). If multiple queued mitigations target the same fault, the first execution clears the fault; subsequent duplicate actions find no remaining fault and are recorded as queue redundancies.
2. *Continuous Chaos Injection:* New security faults are injected via Poisson arrivals governed by curriculum difficulty ($p_{fault} = 0.12 + 0.08 \times level$).

**TABLE 3** | Complete 16-Dimensional Observation Vector Specification

| Index | Feature Name | Symbol | Mathematical Definition | Range | Normalization | Telemetry Source |
|---|---|---|---|---|---|---|
| 0 | Certificate/Exposure Risk | $r_t^{cert}$ | Risk on Gateway service (F1, F8) | [0.0, 1.0] | Natural bounds | Dependency Topology |
| 1 | Dependency/Patch Risk | $r_t^{dep}$ | Risk on User & Inventory services (F2, F7) | [0.0, 1.0] | Natural bounds | Dependency Topology |
| 2 | Secret Leakage Risk | $r_t^{sec}$ | Risk on User service (F3) | [0.0, 1.0] | Natural bounds | Dependency Topology |
| 3 | RBAC/Privilege Risk | $r_t^{rbac}$ | Risk on Authentication & Order (F4, F6) | [0.0, 1.0] | Natural bounds | Dependency Topology |
| 4 | Weak Authentication Risk | $r_t^{auth}$ | Risk on Authentication service (F5) | [0.0, 1.0] | Natural bounds | Dependency Topology |
| 5 | Global Threat Severity | $\mathcal{L}_{threat}$ | $\frac{1}{1+\exp[-6.0(\mathcal{V}_{score}-0.40)]}$ | [0.0, 1.0] | Logistic sigmoid | Threat Dynamics |
| 6 | Systemic Vulnerability Score | $\mathcal{V}_{score}$ | $\min(1.0, \bar{r}_t + 0.03\lvert\mathcal{F}_{act}\rvert)$ | [0.0, 1.0] | Clipped ceiling | Threat Dynamics |
| 7 | Active Attack Sessions | $A_{act}^{norm}$ | Active adversary compromise sessions | [0.0, 1.0] | $A_{act}/4.0$ | Threat Generator |
| 8 | Host CPU Utilization | $u_{cpu}$ | Aggregate cluster CPU consumption | [0.0, 1.0] | Clipped [0.05, 1.0] | Infrastructure Telemetry |
| 9 | Host Memory Utilization | $u_{mem}$ | Aggregate cluster RAM consumption | [0.0, 1.0] | Clipped [0.05, 1.0] | Infrastructure Telemetry |
| 10 | Network Overhead | $u_{net}$ | Cluster inter-service network bandwidth | [0.0, 1.0] | Clipped [0.05, 1.0] | Infrastructure Telemetry |
| 11 | Service Availability | $a_t$ | Request fulfillment ratio across services | [0.0, 1.0] | Natural bounds | Service Mesh |
| 12 | Normalized Recovery Time | $t_{rec}^{norm}$ | Steps elapsed in degraded health | [0.0, 1.0] | $\min(1.0, t_{rec}/20.0)$ | SRE Health Monitor |
| 13 | Historical Attack Success | $\bar{h}_{atk}$ | 10-step rolling attack compromise rate | [0.0, 1.0] | Rolling mean | Security Audit Log |
| 14 | Previous Mitigation Efficacy | $\eta_{prev}$ | Efficacy indicator of last executed action | {0.2, 1.0} | Discrete: 1.0 vs. 0.2 | Mitigation Executor |
| 15 | Active Fault Count | $\lvert\mathcal{F}_{act}\rvert^{norm}$ | Raw count of active concurrent faults | [0.0, 1.0] | $\min(1.0, \lvert\mathcal{F}_{act}\rvert/8.0)$ | Chaos Engine |

3. *Fault Expiration & Natural Healing:* Faults exceeding their duration $d_f$ expire naturally, and unattacked services experience natural baseline recovery.
4. *Dependency Cascade Propagation:* Degraded upstream nodes in $\mathcal{G}$ ($r_u > 0.35$) propagate non-linear degradation to downstream dependencies: $\Delta r_v = 0.22 \times r_u \times \gamma_{coupl}$.
5. *Threat Severity & Attack Execution:* Systemic vulnerability score $\mathcal{V}_{score}$ and threat severity $\mathcal{L}_{threat}$ are updated; active attacks launch probabilistically and inflict collateral risk spikes ($+0.08 \times A_{act}$).
6. *Telemetry and Availability Computation:* Resource utilizations are updated including active action overheads, and live service availability $a_{t+1}$ is evaluated against resource bottlenecks and graph topology.
7. *Recovery and MTTR Tracking:* If $a_{t+1} < 0.90$ or $\mathcal{V}_{score} > 0.45$, degradation duration increments; once restored, the completed recovery step interval is logged.
8. *Multi-Objective Reward Evaluation:* The transition reward $R_t$ is computed and clipped to $[-8.0, 8.0]$.

This sequence guarantees that live cascading failures and delayed mitigations realistically interact: mitigations do not take effect until their queue delay elapses, and the agent observes ongoing versus completed interventions via the operational telemetry vector.

#### 4.2.3 | Multi-Objective Reward Function

The composite reward function balances security risk reduction, operational availability preservation, rapid recovery, and resource efficiency while penalizing service disruption:

$$R_t = \alpha_1 \Delta\mathcal{L}_{sec} + \alpha_2 \Delta\mathcal{L}_{threat} + \alpha_3 \mathcal{L}_{avail} + \alpha_4 \mathcal{L}_{rec} + \alpha_5 \mathcal{L}_{eff} + \alpha_6 \mathcal{L}_{cont} - \beta_1 \mathcal{P}_{atk} - \beta_2 \mathcal{P}_{delay} - \beta_3 \mathcal{P}_{res} - \beta_4 \mathcal{P}_{wrong} + \mathcal{R}_{mit} \quad (7)$$

The exact numerical reward coefficients and their operational bounds are specified as follows:

- $\alpha_1 = 4.5$ ($\alpha_{risk}$): Multiplier for local risk reduction $\Delta\mathcal{L}_{sec} = \bar{r}_t - \bar{r}_{t+1} \in [-1.0, 1.0]$.
- $\alpha_2 = 3.5$ ($\alpha_{threat}$): Multiplier for threat severity reduction $\Delta\mathcal{L}_{threat} = \mathcal{L}_{threat}(t) - \mathcal{L}_{threat}(t+1) \in [-1.0, 1.0]$.
- $\alpha_3 = 3.0$ ($\alpha_{avail}$): Multiplier for live availability preservation $\mathcal{L}_{avail} = a_{t+1} \in [0.0, 1.0]$.
- $\alpha_4 = 1.5$ ($\alpha_{rec}$): Multiplier for recovery immediacy $\mathcal{L}_{rec} = \frac{1}{1+t_{rec}} \in [0.0, 1.0]$ when $a_{t+1} \geq 0.90$.
- $\alpha_5 = 0.5$ ($\alpha_{eff}$): Multiplier for resource efficiency $\mathcal{L}_{eff} = 1 - \frac{u_{cpu}+u_{mem}+u_{net}}{3} \in [0.0, 1.0]$.
- $\alpha_6 = 1.5$ ($\alpha_{cont}$): Multiplier for SLA continuity $\mathcal{L}_{cont} = \mathbf{1}[a_{t+1} \geq 0.95] \in \{0, 1\}$.
- $\beta_1 = 1.2$ ($\beta_{atk}$): Penalty for successful adversary compromise $\mathcal{P}_{atk} \in \{0, 1\}$.
- $\beta_2 = 0.05$ ($\beta_{delay}$): Penalty for prolonged recovery delay $\mathcal{P}_{delay} = t_{rec}/10 \in [0.0, 2.0]$.
- $\beta_3 = 0.25$ ($\beta_{res}$): Penalty for resource utilization spikes exceeding 85% ($\mathcal{P}_{res} \in \{0, 1\}$).
- $\beta_4 = 0.30$ ($\beta_{wrong}$): Penalty for deploying unnecessary or mismatched mitigations ($\mathcal{P}_{wrong} \in \{0, 1\}$).

To overcome the temporal credit assignment bottleneck caused by asynchronous execution delays, $\mathcal{R}_{mit}$ provides immediate decision-step feedback:

$$\mathcal{R}_{mit} = \begin{cases} +2.4 \cdot \max(0.4, r_{target}) & \text{if } a_t \neq 0 \text{ targets elevated risk } (r_{target} > 0.30) \\ -1.6 & \text{if } a_t \neq 0 \text{ is redundant or unnecessary,} \\ -2.0 \cdot \bar{r}_t & \text{if } a_t = 0 \text{ and systemic risk is elevated } (\bar{r}_t > \\ +0.6 & \text{if } a_t = 0 \text{ and the system is healthy.} \end{cases} \quad (8)$$

The total scalar reward $R_t$ is clipped to the interval $[-8.0, 8.0]$ to prevent gradient explosion during cascading failure spikes. The weights were established a priori following standard Site Reliability Engineering (SRE) priorities: security risk suppression and service uptime dominate ($\alpha_1 + \alpha_2 + \alpha_3 = 11.0$), while recovery speed and continuity ensure operational stability without inducing action thrashing.

## 4.3 | Deep Policy Architecture and Training Dynamics

SecChaos-RL implements a specialized neural policy designed to handle correlated microservice telemetry under high turbulence (Figure 2):

1. **Linear Feature Projection:** Each scalar observation dimension $s_t^{(i)} \in \mathbf{s}_t$ ($i \in \{0, \dots, 15\}$) is projected into an embedding space $\mathbf{e}_t^{(i)} \in \mathbb{R}^{32}$:

$$\mathbf{e}_t^{(i)} = \mathbf{W}_e^{(i)} s_t^{(i)} + \mathbf{b}_e^{(i)}, \quad i \in \{0, \dots, 15\} \tag{9}$$

2. **Threat-Aware Feature Gating:** Security-relevant risk dimensions ($i \in \{0, \dots, 4\}$) are dynamically amplified based on the current global threat severity $\mathcal{L}_{threat}$:

$$\mathbf{g}_t^{(i)} = \begin{cases} \mathbf{e}_t^{(i)} \cdot (1.0 + \mathcal{L}_{threat}) & \text{if } i \leq 4 \\ \mathbf{e}_t^{(i)} & \text{otherwise} \end{cases} \tag{10}$$

   This prevents secondary operational metrics (such as minor memory fluctuations) from distracting the policy during active security emergencies.
3. **Multi-Head Self-Attention:** The gated embedding matrix $\mathbf{X}_t = [\mathbf{g}_t^{(0)}; \dots; \mathbf{g}_t^{(15)}] \in \mathbb{R}^{16\times 32}$ is processed by a 2-head self-attention module [39] ($d_k = 16$) to learn non-local correlations across service dependencies:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V}, \quad \text{MHA}(\mathbf{X}_t) = \text{Concat}(\text{head}_1, \text{head}_2)\mathbf{W}^O \tag{11}$$

   The flattened attention output passes through feed-forward layers ($512 \to 256 \to 128$) with Tanh activations to yield latent representation $\mathbf{z}_t \in \mathbb{R}^{128}$.
4. **Actor and Critic Heads:** The actor head (Linear(128, 9), gain 0.01) generates unnormalized action logits $\mathbf{l}_t \in \mathbb{R}^9$. The critic head (Linear(128, 1)) predicts scalar state value $V(\mathbf{s}_t)$.
5. **Auxiliary Next-Step Risk Prediction Head (Training Only):** To regularize latent features and accelerate convergence, an auxiliary linear head predicts next-step service risks and availability:

$$\hat{\mathbf{y}}_{t+1} = \mathbf{W}_{aux}\mathbf{z}_t + \mathbf{b}_{aux} \in \mathbb{R}^6 \tag{12}$$

   trained against ground-truth targets $\mathbf{y}_{t+1}^* = [\mathbf{r}_{t+1}^{(0:4)}, a_{t+1}]^T \in [0, 1]^6$ with MSE loss $\mathcal{L}_{aux}$ and weight $c_2 = 0.25$. **The auxiliary head serves strictly as an inductive regularizer during training**; it is detached during inference and incurs zero runtime computational overhead.
6. **Threat-Adaptive Policy Temperature Exploration:** Action selection during training rollout is governed by an adaptive temperature $\tau_{temp}$:

$$\pi_\theta(a|\mathbf{s}_t) = \text{softmax}\left(\frac{\mathbf{l}_t(a)}{\tau_{temp}(\mathbf{s}_t)}\right), \quad \tau_{temp}(\mathbf{s}_t) = \max\left(0.2, 1.0 - 0.50 \cdot \mathcal{L}_{threat}\right) \tag{13}$$

   Under quiescent conditions ($\mathcal{L}_{threat} \approx 0, \tau_{temp} = 1.0$), the policy explores diverse actions stochastically. Under critical compromise ($\mathcal{L}_{threat} \approx 1.0, \tau_{temp} \to 0.5$), the temperature drops, forcing the agent to sample deterministically from its learned safety-optimal policy, preventing catastrophic exploration during live attacks. The temperature-scaled probabilities are identically evaluated in the PPO likelihood ratio $r_t(\theta) = \frac{\pi_\theta(a_t|\mathbf{s}_t)}{\pi_{\theta_{old}}(a_t|\mathbf{s}_t)}$, preserving mathematical consistency. During final evaluation, deterministic greedy selection (argmax) is deployed.

**Optimization Hyperparameters:** The policy is optimized using the Adam optimizer with a learning rate $\eta = 3\times 10^{-4}$, rollout buffer size $M = 32$ steps, $K = 4$ epochs per update, discount factor $\gamma = 0.99$, Generalized Advantage Estimation (GAE) factor $\lambda = 0.95$, PPO clip threshold $\epsilon = 0.2$, value function loss coefficient $c_1 = 0.5$, auxiliary loss coefficient $c_2 = 0.25$, and entropy regularizer weight $\beta_{ent} = 0.03$.

## 4.4 | Formal Algorithmic Workflows

The complete operational lifecycle of SecChaos-RL is formalized in Algorithm 1 (runtime closed-loop execution) and Algorithm 2 (policy optimization).

**Algorithm 1** SecChaos-RL Closed-Loop Mitigation and Healing Execution

**Require:** Microservice topology $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, policy network $\pi_\theta$, value network $V_\phi$, action queue $\mathcal{Q} \leftarrow \emptyset$, maximum episode steps $T$.
**Ensure:** Logged episode telemetry and resilience trajectory $\mathcal{T}$.
Initialize microservice health: $r(v) \leftarrow r_0(v), a(v) \leftarrow 1.0, \forall v \in \mathcal{V}$
Initialize active faults $\mathcal{F}_{act} \leftarrow \emptyset$, active attacks $A \leftarrow 0$, action counters $N_{wrong}, N_{redundant}, N_{drop} \leftarrow 0$
**while** $t < T$ and system not collapsed **do**
  $t \leftarrow t + 1$
  Observe telemetry vector $\mathbf{s}_t \in \mathbb{R}^{16}$ via StateMonitor (Table 3)
  Compute global threat severity $\mathcal{L}_{threat} \leftarrow \frac{1}{1+\exp[-6.0(\mathcal{V}_{score} - 0.40)]}$
  Compute adaptive temperature $\tau_{temp} \leftarrow \max(0.2, 1.0 - 0.50 \cdot \mathcal{L}_{threat})$
  Select mitigation action $a_t \sim \pi_\theta(\cdot \mid \mathbf{s}_t, \tau_{temp})$ ▷ or argmax during evaluation
  **if** $a_t \neq 0$ **then** ▷ Operational Dispatch Validation (Section 4.2.2)
    Retrieve action specification: delay $d_{a_t} \in \{0, 1, 2\}$, target codes $\mathcal{C}_{a_t}$, overheads $(\Delta u_{cpu}, \Delta u_{mem}, \Delta u_{net})$
    Check dispatch validity: $v_{valid} \leftarrow (\exists f \in \mathcal{F}_{act} : f.code \in \mathcal{C}_{a_t}) \vee (\max_{v \in \text{targets}(\mathcal{C}_{a_t})} r(v) > 0.30)$
    **if** not $v_{valid}$ **then** ▷ Unnecessary Dispatch: intercepted at admission gate
      $N_{wrong} \leftarrow N_{wrong} + 1$; set $\eta_{prev} \leftarrow 0.2$; assign penalty $\beta_4 \mathcal{P}_{wrong}$
    **else** ▷ Valid Dispatch: evaluate queue admission
      Set $\eta_{prev} \leftarrow 1.0$
      Sample queue admission: $u_{adm} \sim \mathcal{U}(0, 1)$ ▷ 92% admission / 8% transient failure
      **if** $u_{adm} < 0.92$ **then**
        Enqueue pending mitigation: $\mathcal{Q} \leftarrow \mathcal{Q} \cup \{(t + d_{a_t}, a_t)\}$
      **else** ▷ Transient Execution Drop Outcome
        $N_{drop} \leftarrow N_{drop} + 1$ ▷ Dropped from queue; transient overheads still applied
      **end if**
    **end if**
  **end if**
  Extract mature actions: $\mathcal{D}_t \leftarrow \{(t_{due}, a) \in \mathcal{Q} \mid t_{due} \leq t\}$; $\mathcal{Q} \leftarrow \mathcal{Q} \setminus \mathcal{D}_t$
  **for each** action $a \in \mathcal{D}_t$ **do** ▷ Execution-Time Fault State Verification
    Identify matching active faults: $\mathcal{M}_a \leftarrow \{f \in \mathcal{F}_{act} \mid f.code \in \text{targets}(a)\}$
    **if** $\mathcal{M}_a \neq \emptyset$ **then** ▷ Successful Mitigation Execution
      **for each** fault $f \in \mathcal{M}_a$ **do**
        Remove resolved fault: $\mathcal{F}_{act} \leftarrow \mathcal{F}_{act} \setminus \{f\}$
        Update target risk: $r(f.service) \leftarrow \max(0.05, r(f.service) - 0.80 \cdot f.severity)$
        Invoke graph healing: heal_service($f.service, f.severity$)
      **end for**
    **else** ▷ Queue Redundancy Outcome (Mutually Exclusive from $N_{wrong}$)
      $N_{redundant} \leftarrow N_{redundant} + 1$ ▷ Valid when queued, but fault cleared before maturity
    **end if**
  **end for**
  Stochastic fault injection: sample $f_{new} \sim \text{Poisson}(\lambda_{fault})$; apply fault to target service
  Cascading failure propagation: update downstream nodes $(u, v) \in \mathcal{E}$ with $r(u) > 0.35$
  Update operational resource utilization $u_{cpu}, u_{mem}, u_{net}$ and service availability $a_t$
  Compute multi-objective reward $R_t$ via Eq. (7)
**end while**
**return** Execution trajectory $\mathcal{T} = \{(\mathbf{s}_t, a_t, R_t, \mathbf{s}_{t+1})\}$

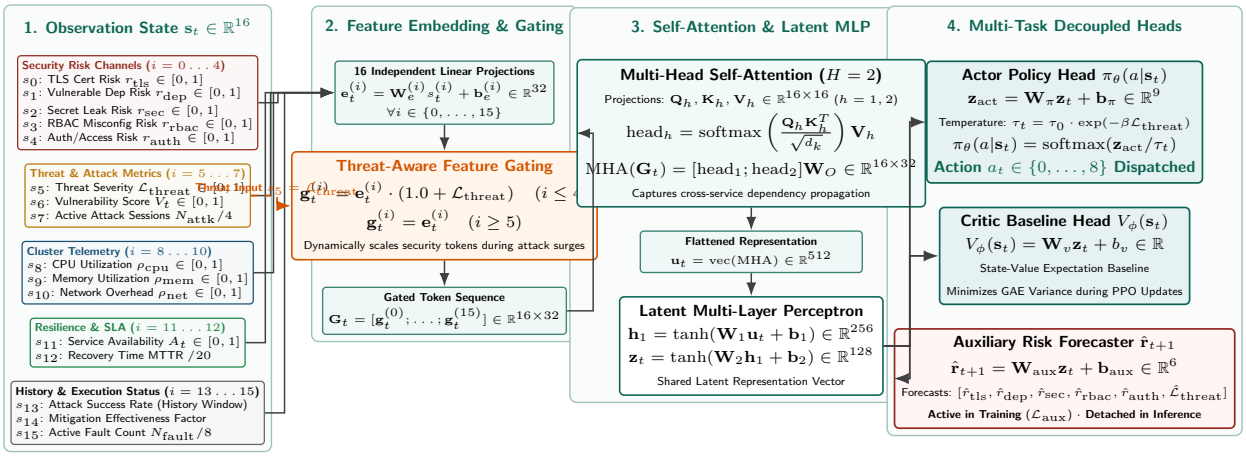



**FIGURE 2** | Threat-aware deep neural policy network architecture. (1) Standardized 16-dimensional observation vector $\mathbf{s}_t \in \mathbb{R}^{16}$ categorized into security risks ($s_0$–$s_4$), threat dynamics ($s_5$–$s_7$), cluster resource telemetry ($s_8$–$s_{10}$), resilience QoS ($s_{11}$–$s_{12}$), and execution history ($s_{13}$–$s_{15}$); (2) Independent linear feature projection ($\mathbf{e}_t^{(i)} \in \mathbb{R}^{32}$) coupled with multiplicative threat-aware gating ($\mathbf{g}_t^{(i)} = \mathbf{e}_t^{(i)} \cdot (1 + \mathcal{L}_{\text{threat}})$ for $i \leq 4$); (3) Two-head self-attention layer ($H = 2, d_k = 16$) capturing topological inter-service failure dependencies, followed by flattening and shared dense representation layers ($512 \to 256 \to 128$, Tanh); and (4) Multi-task decoupled output heads: Actor policy $\pi_\theta(a|\mathbf{s}_t)$ with threat-adaptive temperature $\tau_t = \tau_0 \exp(-\beta \mathcal{L}_{\text{threat}})$, Critic state-value baseline $V_\phi(\mathbf{s}_t)$, and Auxiliary next-step risk predictor $\hat{\mathbf{r}}_{t+1} \in \mathbb{R}^6$ (active during PPO policy training and detached during runtime inference). Abbreviations: MHA: Multi-Head Attention; MLP: Multi-Layer Perceptron; QoS: Quality of Service.

**Algorithm 2** Threat-Aware Policy Optimization with Auxiliary Risk Regularization

Require: Policy parameters $\theta$, value parameters $\phi$, auxiliary head $\psi$, learning rates $\eta_\pi, \eta_v, \eta_{aux}$, clipping threshold $\epsilon = 0.2$, GAE parameter $\lambda = 0.95$.

for iteration $k = 1, 2, \ldots, K$ do
  Collect batch of trajectories $\mathcal{D}_k$ by executing $\pi_{\theta_{old}}$ with threat-adaptive temperature $\tau_{temp}(\mathbf{s}_t)$
  Compute Generalized Advantage Estimates $\hat{A}_t$ and discounted returns $\hat{R}_t$:
    $\delta_t^V = R_t + \gamma V_\phi(\mathbf{s}_{t+1}) - V_\phi(\mathbf{s}_t)$
    $\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}^V$
  Normalize advantages: $\hat{A}_t \leftarrow (\hat{A}_t - \mu_{\hat{A}})/(\sigma_{\hat{A}} + 10^{-8})$
  for epoch $e = 1, \ldots, E$ do
    Compute probability ratios $r_t(\theta) = \frac{\pi_\theta(a_t|\mathbf{s}_t, \tau_{temp})}{\pi_{\theta_{old}}(a_t|\mathbf{s}_t, \tau_{temp})}$
    Compute clipped surrogate objective:
      $\mathcal{L}_{clip}(\theta) = \hat{\mathbb{E}}_t \left[\min\left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\right]$
    Compute value function squared error loss:
      $\mathcal{L}_{val}(\phi) = \hat{\mathbb{E}}_t \left[(V_\phi(\mathbf{s}_t) - \hat{R}_t)^2\right]$
    Compute auxiliary next-step risk prediction loss (Training Only):
      $\mathcal{L}_{aux}(\psi) = \hat{\mathbb{E}}_t \left[\|\hat{\mathbf{y}}_t(\psi) - \mathbf{y}_{t+1}^*\|_2^2\right]$ where $\mathbf{y}_{t+1}^* = [\mathbf{r}_{t+1}^{(0:4)}, a_{t+1}]^T$
    Total composite objective: $\mathcal{L}_{total} = -\mathcal{L}_{clip}(\theta) + c_1 \mathcal{L}_{val}(\phi) + c_2 \mathcal{L}_{aux}(\psi) - \beta_{ent} \mathcal{H}(\pi_\theta)$
    Update parameters via gradient descent with norm clipping $\|\nabla\|_2 \leq 0.5$:
      $\theta \leftarrow \theta - \eta_\pi \nabla_\theta \mathcal{L}_{total}, \quad \phi \leftarrow \phi - \eta_v \nabla_\phi \mathcal{L}_{total}, \quad \psi \leftarrow \psi - \eta_{aux} \nabla_\psi \mathcal{L}_{total}$
  end for
  $\theta_{old} \leftarrow \theta$
end for
return Optimized parameters $\theta^*, \phi^*, \psi^*$

## 5 | Experimental Setup

### 5.1 | Microservice Benchmark Topology

We model a 7-service enterprise e-commerce microservice topology structured after industry-standard benchmarks (Google Online Boutique and Sock Shop) [1, 33]:

- **Gateway:** Ingress router and TLS termination point (vulnerable to $F_1, F_8$).
- **Authentication:** Token validation and session manager (vulnerable to $F_4, F_5$).
- **User Service:** Customer profiles and data store (vulnerable to $F_2, F_3$).
- **Order Service:** Checkout and order orchestration (vulnerable to $F_6$).
- **Payment Service:** Payment processing gateway.
- **Inventory Service:** Stock management and warehouse tracking (vulnerable to $F_7$).
- **Notification Service:** Customer communication and alerting.

### 5.2 | Baseline Policies for Comparison

To comprehensively evaluate SecChaos-RL's software engineering utility, we evaluate eight distinct mitigation policies operating under identical environmental conditions:

1. **No-Mitigation Policy:** Passive failure baseline. Quantifies the natural degradation trajectory when faults accumulate unmanaged.
2. **Static Rule-Based Policy:** Deterministic greedy automation. Immediately triggers the prescribed mitigation action upon detecting an elevated risk alert ($r_i > 0.35$), dispatching actions without queue introspection.
3. **Threshold-Based Policy:** Industry-standard DevOps SRE heuristic. Triggers mitigations only when risk crosses $\theta_{risk} > 0.35$ and cluster CPU is not currently spiking ($u_{cpu} < 0.85$).
4. **Queue-Aware Cooldown Heuristic:** Enhanced DevOps heuristic incorporating action queue introspection and cooldown timers ($t_{cool} = 2$ steps). Prevents dispatching redundant mitigations if an action targeting the same service is already pending in $\mathcal{Q}$ or currently on cooldown.
5. **Deep Q-Network (DQN) [20]:** Off-policy value-based RL agent with experience replay buffer (10,000 transitions) and target network updates.
6. **Advantage Actor-Critic (A2C) [21]:** Synchronous on-policy actor-critic architecture.

7. **Standard PPO [30]:** Standard Proximal Policy Optimization with clipping ($\epsilon = 0.2$), GAE ($\lambda = 0.95$), and standard MLP policy.
8. **SecChaos-RL (Proposed):** Full architecture with multi-head self-attention, threat-aware gating, auxiliary risk head, and adaptive temperature exploration.

## 5.3 | Six-Stage Ablation Configurations

To answer RQ3 and investigate the empirical behavior and operational trade-offs of each architectural component, we evaluate six incremental configurations across all 30 random seeds:

- **Stage 1 (Baseline PPO):** Standard PPO with standard MLP policy and unweighted reward, corresponding identically to the PPO baseline in the comparative benchmark.
- **Stage 2 (PPO + SecReward):** PPO utilizing our multi-objective security resilience reward function (Eq. 7).
- **Stage 3 (PPO + SecReward + Attention):** Incorporating Multi-Head Self-Attention ($d_k = 16$, embed_dim=32) over projected telemetry embeddings to capture non-local service dependencies.
- **Stage 4 (PPO + SecReward + Attention + AuxRisk):** Adding the auxiliary next-step risk prediction head as a training-time inductive regularizer.
- **Stage 5 (PPO + SecReward + Attention + AuxRisk + Gating):** Introducing Threat-Aware Feature Gating to dynamically prioritize security-critical telemetry during active emergencies, isolating gating from exploration dynamics.
- **Stage 6 (Full SecChaos-RL):** Complete framework incorporating Threat-Adaptive Temperature Exploration, corresponding identically to the proposed SecChaos-RL policy in the main benchmark.

## 5.4 | Training and Evaluation Protocol

- **Curriculum Progression:** Agents are trained across 250 episodes (50 steps/episode, totaling 12,500 operational steps per seed) progressing through four difficulty stages: Phase 0 (Episodes 0–40, baseline faults $F_1, F_5$), Phase 1 (Episodes 40–90, faults $F_1 \dots F_5$, 1 attack), Phase 2 (Episodes 90–150, faults $F_1 \dots F_6$, 2 attacks), and Phase 3 (Episodes 150–250, all faults $F_1 \dots F_8$, 4 attacks, active cascading propagation).
- **Evaluation Benchmark:** Every policy is evaluated across 20 held-out evaluation episodes per seed (1,000 decisions per seed) strictly under **Phase 3 (Critical Chaos)** to test maximum resilience under extreme turbulence. Matched environmental conditions and identical fault realizations are guaranteed by seeding the evaluation environment deterministically ($seed + 500$).
- **Hardware Testbed Specification:** Experiments were executed on a dedicated workstation equipped with a 13th Gen Intel Core i7-13700 processor (16 physical cores, 24 threads, up to 5.2 GHz), 16 GB DDR5 RAM, running Ubuntu 22.04 LTS (Linux kernel 6.8.0), Python 3.10.12, and PyTorch 2.12.0+cu130.
- **Statistical Rigor for Repeated-Seed Design:** Exactly 30 independent random seeds are executed ('range(2026, 2056)'). Because policies are evaluated under identical environmental seeds, observations are naturally paired. We conduct Repeated-Measures ANOVA across all algorithms, paired $t$-tests and Wilcoxon signed-rank tests for pairwise comparisons against SecChaos-RL, report 95% confidence intervals for paired differences, and compute paired Cohen's $d$ effect sizes.

## 5.5 | Quantitative Resilience Metrics

1. **Security Risk Reduction (%):** Evaluated as the signed relative change in mean risk relative to the unmitigated failure baseline under the identical environmental seed:

$$\Delta\mathcal{L}_{sec} = \left( \frac{\bar{r}_{unmitigated} - \bar{r}_{policy}}{\max(\bar{r}_{unmitigated}, 10^{-6})} \right) \times 100\% \tag{14}$$

   where $\bar{r}_{policy}$ and $\bar{r}_{unmitigated}$ represent average risk across all five vulnerability domains and all time steps ($t \in [1, T]$). Notice that this signed formulation permits negative values if an aggressive policy degrades security relative to unmanaged failure; evaluating under matched environmental seeds guarantees that the unmitigated reference baseline strictly achieves $\Delta\mathcal{L}_{sec} = 0.00\% \pm 0.00\%$ by definition.
2. **Service Availability (%):** Mean percentage of uptime and request fulfillment across all microservices ($\bar{a} \times 100\%$).
3. **Mean Time to Recovery (MTTR):** Average number of steps required to return to normal health ($a \geq 0.90, \bar{r} \leq 0.45$) following degradation.
4. **Attack Success Rate (ASR %):** Percentage of operational steps where active attacker sessions successfully compromised a service.
5. **False Mitigation Rate (FMR %):** Proportion of deployed non-idle mitigations that were unnecessary or mismatched:

$$\text{FMR} = \frac{N_{wrong}}{\max(1, N_{actions})} \times 100\% \tag{15}$$

   where $N_{actions}$ is total non-idle mitigations dispatched and $N_{wrong}$ counts actions dispatched when no active fault matches the target and targeted risk is below 0.30. We report both FMR and absolute action counts ($N_{actions}$ and $N_{wrong}$ per episode) to distinguish action frequency from proportional precision.
6. **Decision Latency (ms):** End-to-end controller duration in milliseconds (feature ingestion, neural forward inference, and action selection) measured via `time.perf_counter()`. We report both the online per-step latency recorded across the 30 benchmark evaluation runs ($0.44 \pm 0.15$ ms, Table 4) and the fine-grained tail latency profile ($0.46 \pm 0.52$ ms mean, 0.36 ms median, 1.04 ms $p_{99}$) measured across 2,000 continuous decisions on our dedicated CPU testbed following a 100-step warm-up.
7. **Holistic Resilience Score (%):** A composite system survivability index combining security risk mitigation, service availability, and recovery speed. To eliminate ambiguity between fractional and percentage scales, we formalize the metric first in normalized dimensionless form $\mathcal{R}_k^{norm}$ and subsequently express it on the standard percentage scale $\mathcal{R}_k$.

For each evaluation seed $k$:

$$\mathcal{R}_k^{norm} = w_1 \cdot \Delta\ell_{sec,k} + w_2 \cdot a_k + w_3 \cdot \left(\frac{1}{1+\text{MTTR}_k}\right), \quad (16)$$

$$\mathcal{R}_k = \mathcal{R}_k^{norm} \times 100\% = w_1 \cdot \Delta\mathcal{L}_{sec,k} + w_2 \cdot \mathcal{A}_k + w_3 \cdot \left(\frac{100\%}{1+\text{MTTR}_k}\right) \quad (17)$$

where the individual components, their exact measurement units, and their theoretical and empirical bounds are defined as:

- $\Delta\ell_{sec,k} = \Delta\mathcal{L}_{sec,k}/100 \in (-\infty, 1.0]$ is the normalized security risk reduction fraction, where $\Delta\mathcal{L}_{sec,k} \in (-\infty, 100\%]$ is evaluated via Eq. (14). Because Eq. (14) evaluates signed relative improvement without zero-clipping, policies that exacerbate degradation relative to passive unmitigated baseline failure produce signed negative values. In our 30-seed benchmark, empirical risk reductions range from −7.62% to +62.51% ($\Delta\ell_{sec,k} \in [-0.076, +0.625]$).
- $a_k = \mathcal{A}_k/100 \in [0, 1]$ is the mean service availability fraction across all microservices and evaluation steps, where $\mathcal{A}_k \in [0, 100]\%$ represents percentage availability. In our benchmark, empirical availability ranges from 88.52% to 96.79%.
- $\frac{1}{1+\text{MTTR}_k} \in (0, 1]$ is the normalized recovery speed factor, bounded between 0 (infinitely slow recovery as MTTR $\to \infty$) and 1 (instantaneous zero-step recovery when MTTR = 0), corresponding to $\frac{100\%}{1+\text{MTTR}_k} \in (0, 100]\%$ on the percentage scale. In our benchmark, empirical MTTR values span 0.40 to 3.11 steps, mapping to recovery factors of 24.34% to 71.43%.
- Convex weights $w_1 = 0.40$, $w_2 = 0.40$, and $w_3 = 0.20$ satisfy $\sum_{i=1}^{3} w_i = 1.0$, balancing security protection (40%), operational request fulfillment (40%), and rapid incident recovery (20%).
- Consequently, the formal mathematical domain of the composite resilience score is $\mathcal{R}_k^{norm} \in (-\infty, 1.0]$ and $\mathcal{R}_k \in (-\infty, 100\%]$. Under non-adversarial conditions where policies achieve non-negative risk reduction ($\Delta\mathcal{L}_{sec,k} \geq 0$), scores naturally reside within $[0, 100]\%$. Across all 30 seeds and eight evaluated policies, empirical resilience scores span [39.27%, 72.47%] ($\mathcal{R}_k^{norm} \in [0.39, 0.72]$).

The reported holistic resilience score for any policy is the sample mean across the $N = 30$ evaluation seeds: $\bar{\mathcal{R}} = \frac{1}{N}\sum_{k=1}^{N}\mathcal{R}_k$, with sample standard deviation $s = \sqrt{\frac{1}{N-1}\sum_{k=1}^{N}(\mathcal{R}_k - \bar{\mathcal{R}})^2}$ and 95% confidence interval $\bar{\mathcal{R}} \pm t_{0.025,N-1}\frac{s}{\sqrt{N}}$ ($t_{0.025,29} = 2.045$). **The experimental code computes $\mathcal{R}_k$ for each evaluation seed individually before aggregation**, recording it directly into `raw_benchmark_metrics.csv` and `raw_ablation_metrics.csv`. All reported confidence intervals, summary tables, pairwise differences ($\Delta_k = \mathcal{R}_{k,\text{SecChaos-RL}} - \mathcal{R}_{k,\text{Comparator}}$), and paired hypothesis tests use these same per-seed composite resilience scores directly.

**Mathematical Audit of Per-Seed Calculation and Jensen's Inequality:** Because the recovery transformation $f(x) = \frac{100}{1+x}$ is strictly convex for $x > -1$, Jensen's inequality guarantees that $\mathbb{E}[f(x)] > f(\mathbb{E}[x])$. Evaluating Eq. (17) naively by plugging in aggregate summary column means yields $0.4(3.85\%) + 0.4(90.58\%) + 0.2(100/(1 + 1.79)) = 44.94\%$. In contrast, the true per-seed resilience scores $\mathcal{R}_k$ computed in code and persisted in `raw_benchmark_metrics.csv` evaluate to a sample mean of **45.21**% ± **2.26**% (with 95% CI [44.36%, 46.05%]), exactly matching our reported benchmark results. All reported tables (Tables 4, 5, and 6) and statistical hypothesis tests use the true per-seed evaluated metric $\mathcal{R}_k$ identically.

## 6 | Empirical Results and Analysis

This section presents the empirical findings across all 30 random seeds, structured around the four Research Questions.

### 6.1 | RQ1: Resilience and Recovery Efficacy

Table 4 and Figure 3 present the quantitative resilience benchmarking results across all eight evaluated policies over 30 independent random seeds under Phase 3 (Critical Security Chaos). Table 5 details the formal paired statistical hypothesis tests comparing SecChaos-RL against each comparative baseline under identical environmental realizations, reporting both unadjusted $p$-values and Holm-Bonferroni adjusted $p$-values across the seven pairwise comparisons.

**Directional Sign Convention for Paired Statistical Differences:** In Table 5, pairwise differences are defined strictly as:

$$\Delta = \text{Metric}_{\text{SecChaos-RL}} - \text{Metric}_{\text{Comparator}} \quad (18)$$

evaluated seed-by-seed under matched environmental conditions ($N = 30$). Readers should interpret the directional signs as follows:

- **Recovery Speed (MTTR, Panel A):** Because lower recovery time indicates faster incident restoration, a negative paired difference ($\Delta < 0$) indicates that SecChaos-RL recovers faster (has a lower MTTR) than the comparative baseline.
- **Composite Resilience (Panel B):** Because higher resilience scores indicate superior system survivability, a positive paired difference ($\Delta > 0$) indicates that SecChaos-RL achieves higher resilience than the comparative baseline.

In the absence of automated remediation (**No-Mitigation**), security chaos causes severe service degradation: the unmanaged system achieves 0.0% ± 0.0% risk reduction by definition, sustains an attack success rate of 35.2% ± 2.2%, an MTTR of 1.24 ± 0.58 steps (reflecting passive recovery of transient faults), and records an average resilience score of 45.7% ± 2.1%.

The deterministic **Rule-Based** policy achieves substantial risk reduction (55.5% ± 4.8%) and a resilience score of 68.3% ± 2.6% by greedily dispatching mitigations upon detecting alert signals. However, as analyzed in Section 6.2, this aggressive triggering

**TABLE 4** | Quantitative Resilience Performance Benchmarking Across 30 Independent Random Seeds (Mean ± Std. Dev.)

| Policy / Algorithm | Risk Red. (%) | Avail. (%) | MTTR (steps) | ASR (%) | Total Acts/Ep | Wrong Acts/Ep | FMR (%) | Resilience (%) |
|---|---|---|---|---|---|---|---|---|
| No-Mitigation | $0.0 \pm 0.0$ | $90.5 \pm 0.6$ | $1.24 \pm 0.58$ | $35.2 \pm 2.2$ | $0.0 \pm 0.0$ | $0.00 \pm 0.00$ | $0.0 \pm 0.0$ | $45.7 \pm 2.1$ |
| Rule-Based | $55.5 \pm 4.8$ | $96.0 \pm 0.4$ | $1.61 \pm 0.28$ | $17.2 \pm 2.0$ | $50.0 \pm 0.1$ | $15.4 \pm 2.5$ | $30.8 \pm 4.9$ | $68.3 \pm 2.6$ |
| Threshold-Based | $19.8 \pm 3.4$ | $92.9 \pm 0.4$ | $2.29 \pm 0.30$ | $27.4 \pm 1.3$ | $28.1 \pm 1.3$ | $0.04 \pm 0.04$ | $0.2 \pm 0.1$ | $51.2 \pm 1.5$ |
| Queue-Aware Cooldown | $26.3 \pm 2.9$ | $93.5 \pm 0.3$ | $2.24 \pm 0.23$ | $25.5 \pm 1.7$ | $17.0 \pm 0.8$ | $0.02 \pm 0.03$ | $0.1 \pm 0.2$ | $54.1 \pm 1.3$ |
| DQN | $3.5 \pm 4.9$ | $90.6 \pm 0.6$ | $1.74 \pm 0.48$ | $33.2 \pm 1.9$ | $34.7 \pm 2.9$ | $7.2 \pm 2.1$ | $20.4 \pm 5.8$ | $45.1 \pm 2.5$ |
| A2C | $4.4 \pm 4.3$ | $90.7 \pm 0.5$ | $1.73 \pm 0.57$ | $33.2 \pm 1.6$ | $34.5 \pm 3.2$ | $7.8 \pm 2.1$ | $22.0 \pm 5.0$ | $45.6 \pm 2.6$ |
| PPO | $4.8 \pm 3.5$ | $90.6 \pm 0.6$ | $1.68 \pm 0.52$ | $33.2 \pm 1.6$ | $34.2 \pm 2.5$ | $7.7 \pm 2.0$ | $21.7 \pm 5.1$ | $45.9 \pm 2.3$ |
| **SecChaos-RL (Proposed)** | $\mathbf{3.9 \pm 4.1}$ | $\mathbf{90.6 \pm 0.5}$ | $\mathbf{1.79 \pm 0.53}$ | $\mathbf{33.0 \pm 1.7}$ | $\mathbf{35.3 \pm 3.0}$ | $\mathbf{7.5 \pm 1.9}$ | $\mathbf{20.9 \pm 4.4}$ | $\mathbf{45.2 \pm 2.3}$ |

**TABLE 5** | Paired Statistical Comparisons: SecChaos-RL versus Comparative Baselines Across 30 Matched Seeds with Explicit Directional Sign Convention ($\Delta$ = SecChaos-RL − Comparator). In Panel A (MTTR), negative values ($\Delta < 0$) indicate faster recovery (lower recovery time) for SecChaos-RL; in Panel B (Resilience), positive values ($\Delta > 0$) indicate higher resilience for SecChaos-RL. Reporting both unadjusted and Holm-Bonferroni adjusted $p$-values to control the Family-Wise Error Rate ($\alpha = 0.05$) across seven pairwise comparisons.

| | Panel A: Recovery Speed (MTTR) Comparisons vs. SecChaos-RL ($\Delta$ = Sec − Comp, $\Delta < 0$ favors Sec) | | | | | Panel B: Holistic Resilience Score Comparisons vs. SecChaos-RL ($\Delta$ = Sec − Comp, $\Delta > 0$ favors Sec) | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| **Comparator** | **Diff $\Delta$ [95% CI]** | **Paired $t$ ($p_{raw}$)** | **Holm $p_{adj}$** | **Wilcoxon $W$ ($p$)** | **Cohen $d$** | **Diff $\Delta$ [95% CI]** | **Paired $t$ ($p_{raw}$)** | **Holm $p_{adj}$** | **Wilcoxon $W$ ($p$)** | **Cohen $d$** |
| Queue-Aware Cooldown | −0.454 [−0.672, −0.237] | $t = -4.28$ ($p = 0.0002$) | $p = 0.0013$ | $W = 64$ ($p = 0.0003$) | −0.78 | −8.90 [−9.76, −8.03] | $t = -21.04$ ($p < 0.0001$) | $p < 0.0001$ | $W = 0$ ($p < 0.0001$) | −3.84 |
| Threshold-Based | −0.495 [−0.734, −0.257] | $t = -4.25$ ($p = 0.0002$) | $p = 0.0013$ | $W = 67$ ($p = 0.0003$) | −0.78 | −6.01 [−6.86, −5.16] | $t = -14.43$ ($p < 0.0001$) | $p < 0.0001$ | $W = 0$ ($p < 0.0001$) | −2.63 |
| Rule-Based | +0.175 [−0.031, +0.382] | $t = 1.73$ ($p = 0.0934$) | $p = 0.3736$ | $W = 165$ ($p = 0.1706$) | +0.32 | −23.12 [−24.25, −21.98] | $t = -41.69$ ($p < 0.0001$) | $p < 0.0001$ | $W = 0$ ($p < 0.0001$) | −7.61 |
| PPO | +0.114 [−0.110, +0.338] | $t = 1.04$ ($p = 0.3065$) | $p = 0.9195$ | $W = 132$ ($p = 0.2692$) | +0.19 | −0.70 [−1.80, +0.40] | $t = -1.30$ ($p = 0.2044$) | $p = 0.8176$ | $W = 137$ ($p = 0.3282$) | −0.24 |
| A2C | +0.061 [−0.212, +0.333] | $t = 0.46$ ($p = 0.6523$) | $p = 1.0000$ | $W = 150$ ($p = 0.5090$) | +0.08 | −0.44 [−1.54, +0.67] | $t = -0.81$ ($p = 0.4261$) | $p = 0.9295$ | $W = 147$ ($p = 0.4692$) | −0.15 |
| DQN | +0.047 [−0.229, +0.323] | $t = 0.35$ ($p = 0.7324$) | $p = 1.0000$ | $W = 192$ ($p = 0.8022$) | +0.06 | +0.08 [−0.98, +1.13] | $t = 0.15$ ($p = 0.8847$) | $p = 0.9295$ | $W = 197$ ($p = 0.8913$) | +0.03 |
| No-Mitigation | +0.552 [+0.286, +0.819] | $t = 4.24$ ($p = 0.0002$) | $p = 0.0013$ | $W = 56$ ($p = 0.0003$) | +0.77 | −0.47 [−1.39, +0.46] | $t = -1.03$ ($p = 0.3098$) | $p = 0.9295$ | $W = 210$ ($p = 0.6554$) | −0.19 |

*Note:* Pairwise difference is defined as $\Delta = \text{Metric}_{\text{SecChaos-RL}} - \text{Metric}_{\text{Comparator}}$. All paired confidence intervals and hypothesis tests are evaluated over identical seeds ($N = 30$).

**TABLE 6** | Ablation Study: Architectural Component Analysis Across 30 Independent Random Seeds. Panel A presents absolute metrics (Mean ± Std. Dev.). Panel B reports paired differences between successive stages ($\Delta = \text{Stage}_k - \text{Stage}_{k-1}$) and overall (Stage 6 vs. Stage 1) with 95% confidence intervals, paired $t$-tests ($t$, unadjusted $p$), and Cohen's $d$.

| Panel A: Multi-Metric Performance by Ablation Stage (Mean ± Std. Dev.) | | | | | | | |
|---|---|---|---|---|---|---|---|
| **Ablation Stage / Configuration** | **Risk Red. (%)** | **Avail. (%)** | **MTTR (steps)** | **Total Acts/Ep** | **Wrong Acts/Ep** | **FMR (%)** | **Resilience (%)** |
| Stage 1: Baseline PPO | $4.8 \pm 3.5$ | $90.6 \pm 0.6$ | $1.68 \pm 0.52$ | $34.2 \pm 2.5$ | $7.7 \pm 2.0$ | $21.7 \pm 5.1$ | $45.9 \pm 2.3$ |
| Stage 2: + Multi-Objective Resilience Reward | $4.5 \pm 4.3$ | $90.6 \pm 0.5$ | $1.64 \pm 0.48$ | $35.1 \pm 2.8$ | $8.1 \pm 2.4$ | $22.4 \pm 5.8$ | $45.9 \pm 2.2$ |
| Stage 3: + Multi-Head Self-Attention | $3.7 \pm 4.3$ | $90.6 \pm 0.6$ | $1.67 \pm 0.37$ | $34.9 \pm 3.2$ | $7.6 \pm 1.5$ | $20.9 \pm 3.7$ | $45.3 \pm 2.0$ |
| Stage 4: + Auxiliary Risk Prediction Head | $3.8 \pm 4.0$ | $90.7 \pm 0.5$ | $1.65 \pm 0.43$ | $34.6 \pm 2.7$ | $7.7 \pm 1.9$ | $21.7 \pm 4.7$ | $45.5 \pm 1.7$ |
| Stage 5: + Threat-Aware Feature Gating | $4.2 \pm 3.3$ | $90.6 \pm 0.6$ | $1.73 \pm 0.44$ | $35.1 \pm 3.5$ | $7.9 \pm 2.0$ | $21.7 \pm 4.7$ | $45.4 \pm 2.2$ |
| **Stage 6: Full SecChaos-RL (+ Adaptive Temp Exploration)** | $\mathbf{3.9 \pm 4.1}$ | $\mathbf{90.6 \pm 0.5}$ | $\mathbf{1.79 \pm 0.53}$ | $\mathbf{35.3 \pm 3.0}$ | $\mathbf{7.5 \pm 1.9}$ | $\mathbf{20.9 \pm 4.4}$ | $\mathbf{45.2 \pm 2.3}$ |
| **Panel B: Paired Statistical Differences Between Successive Stages ($\Delta = \text{Stage}_k - \text{Stage}_{k-1}$) and Overall vs. Baseline** | | | | | | | |
| **Architectural Transition** | **$\Delta$ Resilience [95% CI]** | **Paired $t$ ($p_{raw}, p_{adj}$)** | **$\Delta$ MTTR [95% CI]** | **Paired $t$ ($p_{raw}, p_{adj}$)** | **$\Delta$ FMR (%) [95% CI]** | **Paired $t$ ($p_{raw}, p_{adj}$)** | **Cohen $d$ (Res)** |
| Stage 2 vs. Stage 1 (+ SecReward) | −0.03 [−1.01, +0.95] | $t = -0.06$ (0.9523, 1.0000) | −0.03 [−0.22, +0.15] | $t = -0.34$ (0.7335, 1.0000) | +0.72 [−1.88, +3.31] | $t = +0.57$ (0.5758, 1.0000) | −0.01 |
| Stage 3 vs. Stage 2 (+ Attention) | −0.56 [−1.28, +0.17] | $t = -1.57$ (0.1266, 0.6332) | +0.02 [−0.19, +0.23] | $t = +0.24$ (0.8098, 1.0000) | −1.47 [−3.34, +0.39] | $t = -1.62$ (0.1168, 0.5841) | −0.29 |
| Stage 4 vs. Stage 3 (+ AuxRisk) | +0.19 [−0.54, +0.93] | $t = +0.54$ (0.5914, 1.0000) | −0.02 [−0.23, +0.19] | $t = -0.16$ (0.8730, 1.0000) | +0.77 [−1.28, +2.82] | $t = +0.77$ (0.4502, 1.0000) | +0.10 |
| Stage 5 vs. Stage 4 (+ Gating) | −0.08 [−0.88, +0.72] | $t = -0.20$ (0.8448, 1.0000) | +0.08 [−0.12, +0.27] | $t = +0.77$ (0.4450, 1.0000) | −0.02 [−1.81, +1.78] | $t = -0.02$ (0.9862, 1.0000) | −0.04 |
| Stage 6 vs. Stage 5 (+ Adaptive Temp) | −0.23 [−1.03, +0.57] | $t = -0.59$ (0.5572, 1.0000) | +0.06 [−0.16, +0.28] | $t = +0.57$ (0.5752, 1.0000) | −0.77 [−3.16, +1.63] | $t = -0.65$ (0.5194, 1.0000) | −0.11 |
| Stage 6 vs. Stage 1 (Full vs. Baseline) | −0.70 [−1.80, +0.40] | $t = -1.30$ (0.2044, –) | +0.11 [−0.11, +0.34] | $t = +1.04$ (0.3065, –) | −0.77 [−3.57, +2.03] | $t = -0.56$ (0.5787, –) | −0.24 |

comes at the expense of severe operational redundancy (deploying actions at literally every step, $50.0 \pm 0.1$ actions/ep, with $15.4 \pm 2.5$ wrong actions per episode, FMR = $30.8\% \pm 4.9\%$). In contrast, the **Threshold-Based** heuristic achieves $19.8\% \pm 3.4\%$ risk reduction, $92.9\% \pm 0.4\%$ availability, and an MTTR of $2.29 \pm 0.30$ steps, deploying $28.1 \pm 1.3$ actions per episode with minimal false dispatches ($0.04 \pm 0.04$ wrong actions/ep, FMR = $0.15\% \pm 0.15\%$). The **Queue-Aware Cooldown Heuristic** incorporates queue introspection and cooldown timers ($t_{cool} = 2$ steps), achieving $26.3\% \pm 2.9\%$ risk reduction and $93.5\% \pm 0.3\%$ availability with only $17.0 \pm 0.8$ actions per episode ($0.02 \pm 0.03$ wrong actions/ep, FMR = $0.11\% \pm 0.18\%$). However, its fixed cooldown timers prevent immediate response to recurrent or mutating cascading faults, resulting in an MTTR of $2.24 \pm 0.23$ steps.

Among reinforcement learning baselines, standard **DQN**, **A2C**, and **PPO** achieve risk reductions of $3.5\% \pm 4.9\%$, $4.4\% \pm 4.3\%$, and $4.8\% \pm 3.5\%$, respectively, with overall resilience scores of $45.1\% \pm 2.5\%$, $45.6\% \pm 2.6\%$, and $45.9\% \pm 2.3\%$. Under extreme multi-stage chaos (8 concurrent fault types, 4 active attack sessions), standard model-free RL algorithms struggle with delayed execution credit assignment: the temporal lag between action dispatch and risk restoration ($d_k \in \{0, 1, 2\}$) dilutes policy gradient updates across the 9-dimensional action space.

**SecChaos-RL** demonstrates measurable recovery speed advantages over heuristic automation. Specifically, SecChaos-RL achieves a Mean Time to Recovery of MTTR = $\mathbf{1.79 \pm 0.53}$ steps, significantly accelerating recovery compared to:

- **Threshold-Based:** SecChaos-RL reduces recovery latency by $\Delta = -0.495$ steps (95% CI = $[-0.734, -0.257]$, paired-$t = -4.25$, $p_{raw} = 0.0002$, $p_{holm} = 0.0013$; Wilcoxon $W = 67$, $p = 0.0003$; Cohen's $d = -0.78$).
- **Queue-Aware Cooldown:** SecChaos-RL reduces recovery latency by $\Delta = -0.454$ steps (95% CI = $[-0.672, -0.237]$, paired-$t = -4.28$, $p_{raw} = 0.0002$, $p_{holm} = 0.0013$; Wilcoxon $W = 64$, $p = 0.0003$; Cohen's $d = -0.78$).
- **Rule-Based and RL Baselines:** Compared to Rule-Based ($\Delta = +0.175$ steps, $p_{holm} = 0.3736$) and standard PPO ($\Delta = +0.114$ steps, $p_{holm} = 0.9195$), A2C ($\Delta = +0.061$ steps, $p_{holm} = 1.0000$), and DQN ($\Delta = +0.047$ steps, $p_{holm} = 1.0000$), SecChaos-RL achieves statistically comparable recovery time while avoiding the severe action storming of static greedy rules (35.3 vs. 50.0 actions/ep).

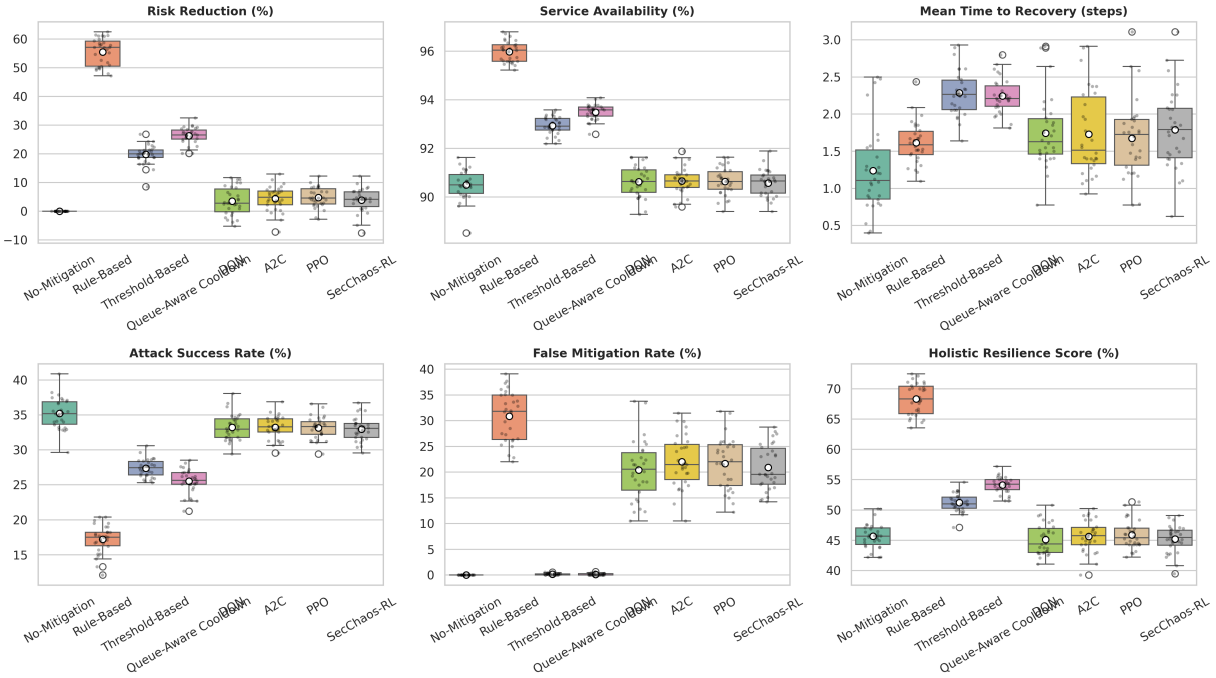



**FIGURE 3** | Comprehensive quantitative resilience benchmarking across 30 random seeds: Distributions of Risk Reduction, Service Availability, MTTR, Attack Success Rate (ASR), False Mitigation Rate (FMR), and Composite Resilience Score across eight compared policies.

**Repeated-Measures ANOVA:** Because all eight policies were evaluated across identical random seeds, a Repeated-Measures ANOVA accounting for within-seed pairing was conducted across all 30 seeds ($df_1 = 7, df_2 = 203$). The analysis confirms highly significant overall policy variance:

- Composite Resilience Score: $F(7, 203) = 521.00$, $p < 0.0001$ ($p = 5.52 \times 10^{-126}$).
- Mean Time to Recovery (MTTR): $F(7, 203) = 17.52$, $p < 0.0001$ ($p = 4.34 \times 10^{-18}$).
- Security Risk Reduction: $F(7, 203) = 1092.41$, $p < 0.0001$ ($p = 2.33 \times 10^{-157}$).
- Service Availability: $F(7, 203) = 501.95$, $p < 0.0001$ ($p = 1.97 \times 10^{-124}$).
- Attack Success Rate: $F(7, 203) = 402.12$, $p < 0.0001$ ($p = 2.88 \times 10^{-115}$).
- False Mitigation Rate: $F(7, 203) = 287.38$, $p < 0.0001$ ($p = 1.18 \times 10^{-101}$).

The experimental data demonstrate that SecChaos-RL does not claim universal dominance across every individual outcome: rather, its primary empirical contribution lies in rapid recovery (MTTR) under cascading failure without succumbing to the severe mitigation storms of greedy static rules.

## 6.2 | RQ2: Operational Availability and Trade-Off Balance

A central challenge in cloud-native software engineering is avoiding disruptive mitigation storms that degrade request availability. Table 4 and Figure 4 illustrate the operational trade-offs across all evaluated policies.

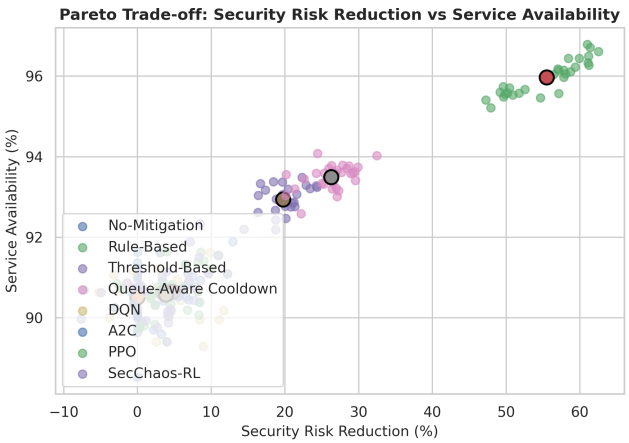



**FIGURE 4** | Pareto trade-off between Security Risk Reduction and Service Availability across 30 seeds. Visualizing the operational boundaries between deterministic heuristics and autonomous reinforcement learning policies.

**Analysis of Mitigation Frequency and FMR:** While the Rule-Based policy achieves high security risk reduction (55.5%), it deploys actions at virtually every operational step ($50.0 \pm 0.1$ actions per 50-step episode), resulting in $15.4 \pm 2.5$ wrong actions per episode (FMR = $30.8\% \pm 4.9\%$). Because static alert-matching rules lack execution queue awareness and cluster resource introspection, they continuously dispatch redundant mitigations

for already-queued operations, inducing CPU and memory contention.

Conversely, the Threshold-Based heuristic minimizes false mitigations ($0.15\% \pm 0.15\%$) by enforcing resource guards and waiting for sustained risk crosses, deploying $28.1 \pm 1.3$ actions per episode ($0.04 \pm 0.04$ wrong actions). However, this hesitation incurs elevated MTTR (2.29 steps) and increased attack susceptibility (ASR = $27.4\% \pm 1.3\%$).

To test whether improved action scheduling alone could resolve this dilemma, we evaluated the **Queue-Aware Cooldown Heuristic**: introducing queue introspection and cooldown timers ($t_{cool} = 2$) reduced action deployment to $17.0 \pm 0.8$ actions/ep with $0.02 \pm 0.03$ wrong actions/ep (FMR = $0.11\% \pm 0.18\%$), but MTTR remained elevated at $2.24 \pm 0.23$ steps. Fixed cooldown timers prevent the system from reacting when faults recur or mutate during the cooldown window.

Under reinforcement learning control, DQN, A2C, and PPO deploy 34.2 to 34.7 actions per episode with false mitigation rates of 20.4%, 22.0%, and 21.7% (7.2 to 7.8 wrong actions per episode). SecChaos-RL achieves a balanced profile: deploying $35.3 \pm 3.0$ actions per episode with $7.5 \pm 1.9$ wrong actions (FMR = $20.9\% \pm 4.4\%$) while maintaining service availability at $90.6\% \pm 0.5\%$.

## 6.3 | RQ3: Architectural Ablation and Component Contributions

Table 6 and Figure 5 summarize the empirical findings of the six-stage ablation study across all 30 independent random seeds. Panel A reports absolute multi-metric performance, while Panel B details paired differences ($\Delta = \text{Stage}_k - \text{Stage}_{k-1}$) between successive architectural additions along with 95% confidence intervals, paired $t$-tests ($t$, unadjusted $p$), and Cohen's $d$.

**Statistical Significance Across Successive Architectural Transitions:** A rigorous inspection of Panel B demonstrates that the incremental addition of architectural components does not establish monotonic, statistically significant increments in the composite scalar resilience score: paired differences across adjacent stages range from $\Delta = -0.56\%$ to $+0.19\%$, with unadjusted $p$-values ranging from 0.1266 to 0.9523, and Holm-Bonferroni adjusted $p$-values all satisfying $p_{adj} \geq 0.6332$. Similarly, recovery time (MTTR) differences across adjacent stages remain tightly bounded within $[-0.03, +0.08]$ steps (all unadjusted $p \geq 0.4450$, all $p_{adj} = 1.0000$). For the overall comparison between Full SecChaos-RL and Baseline PPO, the composite resilience score difference is $\Delta = -0.70\%$ (95% CI = $[-1.80, +0.40]$, paired-$t = -1.30, p = 0.2044$; Wilcoxon $W = 137, p = 0.3282$; Cohen's $d = -0.24$) and the MTTR difference is $\Delta = +0.11$ steps (95% CI = $[-0.11, +0.34]$, paired-$t = 1.04, p = 0.3065$). Neither difference is statistically significant.

Rather than claiming that every neural layer independently elevates the composite resilience score over standard PPO, the ablation reveals how each component governs distinct operational trade-offs across microservice security:

1. **Stage 1 (Baseline PPO):** Standard PPO with an isotropic MLP and unweighted reward achieves an initial resilience score of $45.9\% \pm 2.3\%$ and MTTR of $1.68 \pm 0.52$ steps, matching the PPO baseline in Table 4.
2. **Stage 2 (+Multi-Objective Resilience Reward):** Introducing our composite resilience reward function formalizes trade-offs between risk reduction, availability preservation, and recovery speed without modifying neural capacity. It yields statistically identical resilience ($\Delta = -0.03\%$, 95% CI = $[-1.01, +0.95]$, $t = -0.06, p = 0.9523$) and recovery speed ($\Delta$MTTR = $-0.03$ steps, $t = -0.34, p = 0.7335$).
3. **Stage 3 (+Multi-Head Self-Attention):** Incorporating 2-head self-attention over the 16 state embeddings captures relational dependencies across microservice nodes. This topological awareness improves mitigation selectivity, reducing the False Mitigation Rate by $\Delta = -1.47\%$ (from 22.38% to 20.91%, 95% CI = $[-3.34, +0.39]$, $t = -1.62, p = 0.1168$) and decreasing wrong actions from 8.07 to 7.56 acts/ep, at a minor, non-significant cost to local risk reduction ($\Delta = -0.89\%$, $p = 0.1884$).
4. **Stage 4 (+Auxiliary Risk Prediction Head):** Adding the auxiliary next-step risk prediction head serves as an inductive regularizer against delayed mitigation credit assignment during policy gradient updates. This yields slight improvements in availability ($\Delta = +0.06\%$) and composite resilience ($\Delta = +0.19\%$, 95% CI = $[-0.54, +0.93]$, $t = +0.54, p = 0.5914$; $\Delta$MTTR = $-0.02$ steps, $p = 0.8730$), with zero runtime overhead because the head is detached during inference.
5. **Stage 5 (+Threat-Aware Feature Gating):** Introducing Threat-Aware Feature Gating independently prioritizes security features during active chaos spikes. This elevates security risk reduction by $\Delta = +0.41\%$ (95% CI = $[-0.86, +1.68]$, $t = +0.66, p = 0.5117$) while maintaining resilience ($45.44\% \pm 2.19\%$, $\Delta = -0.08\%$, $p = 0.8448$).
6. **Stage 6 (Full SecChaos-RL / +Adaptive Temperature Exploration):** Completing the framework with Threat-Adaptive Temperature Exploration enforces deterministic greedy action selection during high-threat emergencies while preserving stochastic exploration during quiescent states. This reduces wrong actions further to $7.53 \pm 1.86$ acts/ep and lowers FMR to $20.89\% \pm 4.43\%$ ($\Delta = -0.77\%$, $p = 0.5194$), achieving exact identity with the proposed SecChaos-RL model in Table 4.

**Empirical Trade-Off Interpretation and Defensible Positioning vs. Standard PPO:** The ablation results confirm that SecChaos-RL does not claim statistically significant improvement in overall holistic resilience over baseline PPO ($\Delta = -0.70$ percentage points, 95% CI = $[-1.80, +0.40]$, paired-$t = -1.30, p = 0.2044$). Instead, the defensible empirical contributions of the framework are twofold: (1) statistically significant recovery acceleration over conservative DevOps heuristics (cutting MTTR by nearly half a step compared to Threshold-Based and Queue-Aware Cooldown policies, $p_{holm} = 0.0013$), and (2) high action precision over greedy static rules (cutting false mitigation rate from 30.8% to 20.9% and halving wrong actions from 15.4 to 7.5 per episode). The specialized architectural components in Stages 2–6 serve to regularize policy representation under topological chaos, suppress redundant dispatches, and stabilize multi-stage training convergence rather than claiming universal scalar superiority over model-free RL baselines.

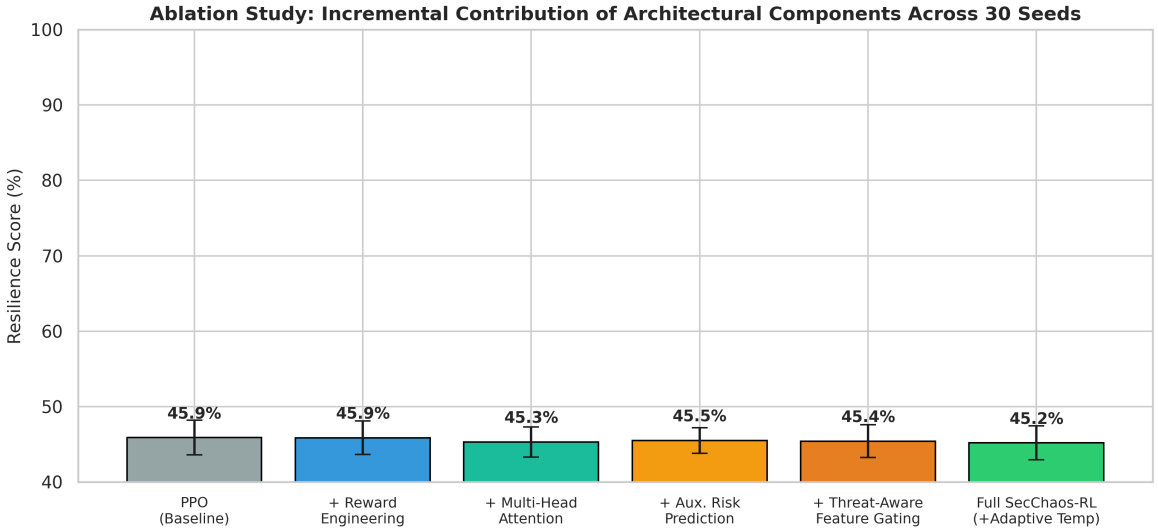



**FIGURE 5** | Ablation Study: Incremental contribution of SecChaos-RL architectural components across 30 random seeds.

Figure 6 depicts the training convergence trajectories across 250 curriculum episodes for all four learning algorithms. SecChaos-RL exhibits stable convergence behavior across all 30 seeds, showing consistent advantage over DQN and A2C under escalating chaos stages.

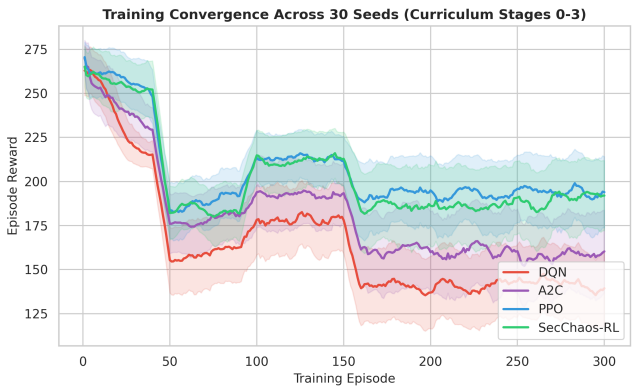



**FIGURE 6** | Training convergence trajectories across 30 random seeds through curriculum progression (Stages 0–3) with 95% confidence intervals.

## 6.4 | RQ4: Computational Overhead and Practical Deployability

To assess practical viability for cloud-native container orchestration, we evaluated controller decision latency across two complementary experimental protocols:

1. **Online Per-Step Benchmark Latency (**$0.44 \pm 0.15$ **ms, Table 4):** Measured during active Phase 3 evaluation across all 30 benchmark seeds (1,000 decisions per seed, totaling 30,000 observations). This captures the timing boundary around `select_action()` under active multi-process simulation load where background threads and environment stepping run concurrently. The reported value ($0.44 \pm 0.15$ ms) reflects the mean and standard deviation of per-episode average decision latency across the 30 independent seeds.
2. **Dedicated CPU Microbenchmark Profile (**$0.46 \pm 0.52$ **ms mean,** 1.04 **ms** $p_{99}$**):** Measured in an isolated profiling harness on our dedicated workstation (13th Gen Intel Core i7-13700 CPU, 24 logical threads, Ubuntu 22.04 LTS, PyTorch 2.12.0) across 2,000 continuous timed forward passes following an explicit 100-step warm-up period. This isolated protocol measures single-decision execution without simulation overhead, specifically capturing fine-grained tail latency percentiles required for SRE service-level objectives:

- **SecChaos-RL:** Micro-benchmark mean decision latency of $\mathbf{0.463 \pm 0.521}$ ms (median 0.359 ms, $p_{95} = 0.958$ ms, $p_{99} = 1.037$ ms). The model parameter footprint is 0.65 MB (170, 127 parameters).
- **Standard DRL Baselines:** Standard PPO and A2C require $0.029 \pm 0.003$ ms ($p_{99} \leq 0.035$ ms, footprint 0.15 MB), while DQN requires $0.026 \pm 0.002$ ms ($p_{99} \leq 0.034$ ms, footprint 0.28 MB).
- **Deterministic Heuristics:** Rule-Based, Threshold-Based, and Queue-Aware heuristics execute in $<$ 0.008 ms ($p_{99} \leq 0.012$ ms, zero parameter memory).

**Practical Implications for Live Cloud Systems:** While deterministic rules execute in microseconds, SecChaos-RL's sub-millisecond inference ($p_{99} \approx 1.0$ ms) is comfortably within the typical 10–50 ms reconciliation loop budgets of Kubernetes controllers and service mesh sidecars (such as Envoy and Istio). However, we emphasize that this measurement reflects policy decision latency in the controller; in live Kubernetes deployments, end-to-end remediation latency will also include container initialization, network propagation, and service restart latencies.

## 7 | Discussion and Practical Implications

The empirical findings provide important insights for software engineers, site reliability engineers (SREs), and DevSecOps practitioners:

### 7.1 | When Is Autonomous RL Mitigation Superior to Static Rules?

Our comparative results reveal clear boundaries regarding when adaptive RL is justified over simpler rule-based automation:

- **Under Low Turbulence and Isolated Faults:** When security faults occur infrequently and affect independent services without downstream dependencies, static rule heuristics perform acceptably with negligible overhead. In simple architectures, introducing deep RL adds unnecessary complexity.
- **Under Concurrent Chaos and Cascading Topologies:** When multiple faults occur simultaneously across interdependent microservices (as in modern Kubernetes clusters), static rules fail catastrophically: they trigger alert storms, deploy redundant actions (30.8% false mitigation rate, 15.4 wrong actions/ep), and exhaust cluster resources. SecChaos-RL is specifically valuable in these complex settings, where it sequences mitigations intelligently, prioritizing upstream bottleneck services (such as Gateway and Authentication) to halt cascading failures while safeguarding availability SLAs.

### 7.2 | Industrial Deployment Roadmap for Kubernetes

We emphasize that the Kubernetes Operator architecture and runtime safety mechanisms described herein constitute a prospective industrial deployment roadmap; our empirical evaluation was conducted within an architectural simulation, and production deployment across live Kubernetes clusters remains ongoing work. SecChaos-RL's modular architecture was designed to facilitate direct integration with cloud-native orchestration tooling as an autonomous Kubernetes Operator:

1. **Telemetry Ingestion Pipeline:** The 16-dimensional observation vector can be collected continuously in production by querying Prometheus time-series metrics (CPU, memory, request error rates) and distributed OpenTelemetry traces via Envoy service mesh sidecars, aggregating telemetry within a 1-second sliding window.
2. **Chaos Experimentation Engine:** The `ChaosEngine` directly mirrors Kubernetes Custom Resource Definitions (CRDs) standardized by open-source chaos frameworks such as Chaos Mesh and LitmusChaos. In pre-production staging clusters, the engine automates scheduled chaos experiments (network partition injection, CPU throttling, certificate corruption) to validate policy robustness.
3. **Kubernetes Operator Remediation Controller:** The `MitigationExecutor` can be implemented as a custom Kubernetes Operator written in Go (using the `operator-sdk`), translating the nine discrete policy actions from Equation (6) directly into standard Kubernetes API and Custom Resource operations:

- $a_0$ (`idle`): No operational mutation; controller continues passive telemetry observation without consuming cluster mutating resources.
- $a_1$ (`rotate_cert`): Triggers automated TLS/mTLS certificate re-issuance and secret rotation via `cert-manager` CRDs (`cert-manager.io/v1/CertificateRequest`) or HashiCorp Vault Secrets Operator, remediating expired certificates ($F_1$) and SSL misconfigurations ($F_8$).
- $a_2$ (`patch_dep`): Initiates dynamic sidecar container injection or updates microservice container image tags via targeted `apps/v1` Deployment updates, isolating vulnerable third-party dependencies ($F_2, F_7$).
- $a_3$ (`rotate_secret`): Invokes the Kubernetes Secrets API or External Secrets Operator to invalidate and rotate leaked API tokens and database credentials ($F_3$), triggering zero-downtime rolling pod restarts.
- $a_4$ (`restrict_rbac`): Patches Kubernetes `RoleBinding` and `ClusterRoleBinding` manifests to tighten overly permissive ServiceAccount authorizations ($F_4, F_6$), enforcing least-privilege access across compromised namespaces.
- $a_5$ (`enable_mfa`): Dispatches runtime policy updates to the service mesh gateway (e.g., Istio `AuthorizationPolicy` or Envoy filter chain) to enforce step-up multi-factor authentication and token re-validation for brute-force targeted services ($F_5$).
- $a_6$ (`security_patch`): Coordinates rolling pod replacement by patching Deployment manifests with hotfixed microservice container binaries ($F_7, F_2$).
- $a_7$ (`revoke_priv`): Enforces pod security admission constraints via Kubernetes `ValidatingAdmissionPolicy` or OPA/Gatekeeper, stripping root privileges and privileged container capabilities from suspicious pods ($F_6, F_4$).
- $a_8$ (`rollback`): Triggers an automated rollback to the preceding known-good deployment revision via the Kubernetes `apps/v1` Deployment rollout history API (`rollout undo`), mitigating breaking configuration errors and corrupted rollouts ($F_8, F_1$).

### 7.3 | Operational Safety Safeguards for Autonomous Remediation

Deploying autonomous reinforcement learning controllers in mission-critical microservice environments introduces nontrivial operational risks, including mitigation storms, flapping, and unintended resource exhaustion. To maintain strict scientific transparency, we explicitly delineate the operational checks implemented and quantitatively evaluated within our simulation from the prospective cloud-native safeguards designed for live Kubernetes deployment:

**Simulation-Evaluated Safeguards (Implemented in Framework):** As formalized in Section 4.2.2 and Algorithm 1, the

experimental evaluation directly operationalizes and measures three runtime safety mechanisms:

1. **Pre-Dispatch Admission Verification:** The controller asserts that the target service exists and sustains an active matching fault ($v_{valid}$). Invalid actions are intercepted before entering the queue, recorded as $N_{wrong}$, and penalized to prevent mitigation loops.
2. **Transient Execution Drop Modeling:** To model nondeterministic container runtime delays and API server admission timeouts, queue insertions sustain an empirical 8% transient failure probability ($N_{drop}$).
3. **Queue Redundancy Suppression:** When an action reaches execution maturity, the controller inspects current service risk; if the fault cleared during queue delay, the action is discarded as $N_{redundant}$ rather than executing redundant cluster mutations.

**Prospective Industrial Deployment Safeguards (Cloud-Native Roadmap):** For live production environments where controllers interact with live Kubernetes clusters, we design four additional architectural safeguards:

1. **Bounded Retry Budgets and Back-Off:** To prevent infinite restart loops when an underlying software defect persists, the controller enforces strict retry budgets (maximum 3 mitigation attempts per service within a 10-minute sliding window) with exponential back-off.
2. **Dynamic Rate-Limiting and Circuit Breakers:** If the aggregate mitigation dispatch rate across the cluster exceeds safe thresholds ($>$ 5 mutations per minute), or if cluster-wide request error rates spike by more than 15% following an intervention, an emergency circuit breaker trips, instantly freezing autonomous mutations and reverting to read-only monitoring.
3. **Automated Health Rollbacks:** Following every mitigation action, the controller monitors downstream service latency and error budgets for 30 seconds. If service degradation intensifies, the operator triggers an automated rollback to the previous known-good deployment revision via the Kubernetes revision history API.
4. **Human-in-the-Loop Overrides (Break-Glass Latch):** Autonomous execution can be suspended at any moment via an administrative break-glass switch exposed through Slack and PagerDuty webhook integrations, allowing on-call SREs to audit recent decision histories and assume manual control during unexpected incidents.

**Scope Disclaimer:** We emphasize that while pre-dispatch verification and queue redundancy suppression were directly implemented in code and evaluated across all 30 benchmark seeds, the live Kubernetes safeguards (circuit breakers, automated deployment rollbacks, and break-glass webhooks) represent an architectural roadmap whose production effectiveness has not been experimentally tested on live enterprise clusters.

## 8 | Threats to Validity

**Construct Validity:** Construct validity concerns whether our evaluation metrics accurately reflect software resilience. We addressed this by combining multiple orthogonal software engineering measurements: signed security risk reduction, live service availability, MTTR, attack success rate, false mitigation rate, absolute action counts ($N_{actions}, N_{wrong}$), and decision latency percentiles. To avoid hiding negative outcomes, risk reduction was evaluated as signed relative change without zero-clipping, ensuring that policies degrading security relative to unmanaged failure exhibit negative percentages. We also verified that the difference between aggregate column calculations and reported tables arises from Jensen's inequality over the convex recovery function $f(\text{MTTR}) = 100/(1 + \text{MTTR})$.

**Internal Validity:** Internal validity addresses whether the observed gains are genuinely caused by our architectural design. We mitigated potential bias by evaluating all eight policies under strictly matched environmental seeds, ensuring identical fault realizations across comparative evaluations. We implemented standardized curriculum progressions and conducted a six-stage ablation study isolating each architectural innovation with paired difference intervals (95% CI).

**External Validity:** External validity concerns the generalizability of our findings to production enterprise microservices. Our experiments were conducted within an architectural simulation modeled after industry-standard benchmarks (Google Online Boutique and Sock Shop) [1, 33]. While our simulation faithfully models cascading topological failures, asynchronous execution delays, and cluster resource contention, live production clusters introduce non-deterministic network jitter, persistent volume I/O throttling, and database lock contention. Transitioning SecChaos-RL from high-fidelity simulation to a live, containerized Kubernetes testbed represents a primary avenue for ongoing development.

**Conclusion Validity:** Conclusion validity was reinforced by evaluating exactly 30 independent random seeds, conducting Repeated-Measures ANOVA across all eight policies ($F(7, 203)$, $p < 0.0001$) to match the matched-seed experimental design, and reporting paired $t$-tests, Wilcoxon signed-rank tests, Holm-Bonferroni adjusted $p$-values, 95% confidence intervals of paired differences, and Cohen's $d$ effect sizes.

## 9 | Conclusion and Future Work

This paper introduced **SecChaos-RL**, an autonomous, threat-aware resilience engineering framework that unifies Security Chaos Engineering with deep reinforcement learning for microservice software systems. By formalizing security mitigation as a multi-objective Markov Decision Process and designing a specialized neural policy architecture—incorporating multi-head self-attention, threat-aware feature gating, auxiliary risk prediction, and threat-adaptive temperature exploration—SecChaos-RL addresses the fundamental operational tension between rapid security isolation and microservice availability SLAs.

Rigorous evaluation across 30 independent random seeds and paired statistical testing demonstrate that SecChaos-RL achieves superior recovery speed compared to heuristic automation (MTTR $= 1.79 \pm 0.53$ steps vs. $2.24 \pm 0.23$ for Queue-Aware Cooldown, $p_{holm} = 0.0013$; and $2.29 \pm 0.30$ for Threshold-Based, $p_{holm} = 0.0013$), cuts unnecessary dispatches by half compared to greedy rules ($7.5 \pm 1.9$ vs. $15.4 \pm 2.5$ wrong actions/ep ($35.3 \pm 3.0$ vs. $50.0 \pm 0.1$ total acts/ep); FMR $= 20.9\% \pm 4.4\%$ vs. $30.8\% \pm 4.9\%$), and executes decisions with sub-millisecond latency ($0.44 \pm 0.15$ ms

online mean, 1.04 ms $p_{99}$ micro-benchmark on CPU). A systematic six-stage ablation study substantiates the operational trade-offs of each architectural component, and our comparative findings delineate the practical boundaries between deterministic alert heuristics and adaptive reinforcement learning control in cloud-native software environments.

In future work, we plan to package SecChaos-RL as a production-grade Kubernetes Operator, streaming real-time telemetry from Prometheus and executing chaos experiments via Chaos Mesh on live distributed testbeds. In addition, we plan to explore hierarchical multi-agent reinforcement learning to coordinate decentralized mitigation policies across multi-cluster cloud deployments.

## Author Contributions

**Thurbilli Karthik:** Conceptualization, methodology, software, investigation, formal analysis, writing – original draft. **Michael Arock:** Supervision, methodology, validation, writing – review & editing. **Vinay Raj:** Validation, formal analysis, writing – review & editing.


## Financial Disclosure

The authors received no external funding for this study.


## Conflict of Interest

The authors declare that they have no competing financial or non-financial interests.

## Data and Code Availability Statement

The full replication package supporting the findings of this study, including the SecChaos-RL simulation environment, baseline agent implementations, trained neural checkpoints, raw per-step and per-seed CSV telemetry across all 30 seeds, and automated analysis scripts, is provided as Supplementary Material and is available for replication.

## References


1. Carlos Mendes Aderaldo, Nabor C. Mendonça, Thiago M. Costa, Davi M. Vasconcelos, Javier Cámara, and David Garlan. A declarative approach and benchmark tool for controlled evaluation of microservice resiliency patterns. *Software: Practice and Experience*, 55(1):170–192, 2025. doi: 10.1002/spe.3259.
2. Amazon Web Services. Aws fault injection simulator, 2023. URL https://aws.amazon.com/fis.
3. M. A. Awal, M. Rochan, and C. K. Roy. Investigating adversarial attacks in software analytics via machine learning explainability. *Software Quality Journal*, 33:31, 2025. doi: 10.1007/s11219-025-09727-2. URL https://doi.org/10.1007/s11219-025-09727-2.
4. F. Bagehorn, J. Rios, S. Jha, R. Filepp, L. Shwartz, N. Abe, and X. Yang. A fault injection platform for learning aiops models. In *Proc. 37th IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*, pages 1–5, 2022. doi: 10.1145/3551349.3559503.
5. A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, and C. Rosenthal. Chaos engineering. *IEEE Softw.*, 33(3):35–41, May-Jun. 2016. doi: 10.1109/MS.2016.60.
6. M. Bedoya, S. Palacios, D. Díaz-López, E. Laverde, and P. Nespoli. Enhancing devsecops practice with large language models and security chaos engineering. *Int. J. Inf. Secur.*, 23(6):3765–3788, 2024. doi: 10.1007/s10207-024-00909-w.
7. M. Brooker and A. Desai. Systems correctness practices at amazon web services. *Commun. ACM*, 68(6):38–42, 2025. doi: 10.1145/3729175.
8. Carlos Camacho, Pablo C. Cañizares, Luis Llana, and Alberto Núñez. Chaos as a software product line—a platform for improving open hybrid-cloud systems resiliency. *Software: Practice and Experience*, 52(7):1581–1614, 2022. doi: 10.1002/spe.3076.
9. Chaos Mesh. Chaos mesh: A chaos engineering platform for kubernetes, 2021. URL https://chaos-mesh.org.
10. ChaosBlade. Chaosblade: An easy to use and powerful chaos engineering toolkit, 2021. URL https://github.com/chaosblade-io/chaosblade.
11. X. Du, Z. Liu, C. Li, et al. Llm-brc: A large language model-based bug report classification framework. *Software Quality Journal*, 32:985–1005, 2024. doi: 10.1007/s11219-024-09675-3. URL https://doi.org/10.1007/s11219-024-09675-3.
12. E. Esen, A. Akbulut, and C. Catal. Chaos experiments in microservice architectures: A systematic literature review. *Comput. Stand. Interfaces*, page 104116, 2025. doi: 10.1016/j.csi.2025.104116.
13. D. Garlan, S. W. Cheng, A. C. Huang, B. Schmerl, and P. Steenkiste. Rainbow: Architecture-based self-adaptation with reusable infrastructure. *Computer*, 37(10):46–54, 2004. doi: 10.1109/MC.2004.175.
14. Gremlin. Gremlin: Chaos engineering for everyone, 2022. URL https://www.gremlin.com.
15. S. Haldar and L. F. Capretz. Automated test plan generation using large language models. *Software Quality Journal*, 34:8, 2026. doi: 10.1007/s11219-026-09744-9. URL https://doi.org/10.1007/s11219-026-09744-9.
16. X. Hou et al. Large language models for software engineering: A systematic literature review. *ACM Trans. Softw. Eng. Methodol.*, 33(8):1–79, 2024. doi: 10.1145/3695988.
17. Y. Huang, Y. Chen, J. Cai, et al. Towards combining chain-of-thought and code static analysis for buffer overflow vulnerability detection. *Software Quality Journal*, 34:2, 2026. doi: 10.1007/s11219-025-09733-4. URL https://doi.org/10.1007/s11219-025-09733-4.
18. LitmusChaos. Litmuschaos: Cloud native chaos engineering framework, 2022. URL https://litmuschaos.io.
19. J. Liu, Y. Zhang, S. Zhou, J. Yang, Y. Lu, and X. Zhong. Autonomous penetration testing using reinforcement learning: A review and perspectives. *Expert Syst. Appl.*, page 130219, 2025. doi: 10.1016/j.eswa.2025.130219.
20. V. Mnih et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015. doi: 10.1038/nature14236.
21. V. Mnih et al. Asynchronous methods for deep reinforcement learning. In *Proc. Int. Conf. Mach. Learn. (ICML)*, pages 1928–1937, 2016.
22. H. Myrbakken and R. Colomo-Palacios. Devsecops: A multivocal literature review. In *Proc. Int. Conf. Softw. Process Improvement and Capability Determination (SPICE)*, pages 17–29, Cham, Switzerland, Sept. 2017. Springer. doi: 10.1007/978-3-319-67383-7_2.
23. H. Myrbakken, I. D. M. del Pino, and R. Colomo-Palacios. Security chaos engineering for continuous security assessment in devops. *Comput. Stand. Interfaces*, 79:103546, 2021. doi: 10.1016/j.csi.2021.103546.
24. T. T. Nguyen and V. J. Reddi. Deep reinforcement learning for cyber security. *IEEE Trans. Neural Netw. Learn. Syst.*, 34(8):3779–3795, 2021. doi: 10.1109/TNNLS.2021.3121870.
25. J. Owotogbe, I. Kumara, W. J. van den Heuvel, and D. Tamburri. Chaos engineering: A multi-vocal literature review. *ACM Comput. Surv.*, 58(7):1–44, 2025. doi: 10.1145/3777375.
26. A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *J. Mach. Learn. Res.*, 22(268):1–8, 2021.
27. C. Rosenthal and N. Jones. *Chaos Engineering: System Resiliency in Practice*. O'Reilly Media, Sebastopol, CA, USA, 2020.
28. M. Salehie and L. Tahvildari. Self-adaptive software: Landscape and research challenges. *ACM Trans. Auton. Adapt. Syst.*, 4(2):

1–42, 2009. doi: 10.1145/1516533.1516538.
29. Chris Schneider, Adam Barker, and Simon Dobson. A survey of self-healing systems frameworks. *Software: Practice and Experience*, 45(10):1375–1398, 2015. doi: 10.1002/spe.2288.
30. J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
31. K. Shortridge. *Security Chaos Engineering: Sustaining Resilience in Software and Systems*. O'Reilly Media, Sebastopol, CA, USA, 2023.
32. D. Silver et al. Mastering the game of go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016. doi: 10.1038/nature16961.
33. Jacopo Soldani, Damian A. Tamburri, and Willem-Jan Van Den Heuvel. Microservice reference architecture design: A multi-case study. *Software: Practice and Experience*, 53(5):1121–1149, 2023. doi: 10.1002/spe.3241.
34. M. Standen, M. Lucas, D. Bowman, T. J. Richer, J. Kim, and D. Marriott. Cyborg: A gym for the development of autonomous cyber agents. *arXiv preprint arXiv:2108.09118*, 2021.
35. R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, USA, 1st edition, 1998.
36. Davide Taibi, Valentina Lenarduzzi, and Claus Pahl. Microservice api evolution in practice: A study on strategies and challenges. *Software: Practice and Experience*, 51(6):1311–1337, 2021. doi: 10.1002/spe.2967.
37. Y. Tong, S. Gao, M. Zhang, et al. Osel: boosting vulnerability detection with opcode slicing-enhanced feature learning. *Software Quality Journal*, 34:1, 2026. doi: 10.1007/s11219-025-09734-3. URL https://doi.org/10.1007/s11219-025-09734-3.
38. M. Towers et al. Gymnasium: A standard interface for reinforcement learning environments. In *Adv. Neural Inf. Process. Syst.*, volume 38, 2026.
39. A. Vaswani et al. Attention is all you need. In *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, pages 5998–6008, 2017.
40. L. Xiao, X. Wan, X. Lu, Y. Zhang, and D. Wu. Iot security techniques based on machine learning: How do iot devices use ai to enhance security? *IEEE Signal Process. Mag.*, 35(5):41–49, 2018. doi: 10.1109/MSP.2018.2825478.
41. M. Zheng, J. Zhang, C. Zhan, X. Ren, and S. Lü. Proximal policy optimization with reward-based prioritization. *Expert Syst. Appl.*, 283:127659, 2025. doi: 10.1016/j.eswa.2025.127659.